ObraYa

Documentacion Completa de Procesos y Analisis de Variantes

Plataforma de abastecimiento de materiales de construccion

Agente inteligente WhatsApp-to-WhatsApp

Fecha: Abril 2026

Version: 1.0

Clasificacion: Confidencial

PARTE 1

Mapa Completo de Procesos

ObraYa -- Mapa Completo de Procesos

1. Vision General

1.1 Que es ObraYa

ObraYa es una plataforma de abastecimiento de materiales de construccion que opera 100% por WhatsApp. A diferencia de un marketplace con precios estaticos, ObraYa funciona como un agente inteligente ("Nico") que contacta proveedores en tiempo real, solicita cotizaciones por WhatsApp, compara respuestas y permite al comprador elegir la mejor opcion -- todo sin salir de la conversacion. El residente de obra pide material desde el campo, Nico hace el trabajo de llamar a 20 proveedores, y en minutos tiene una comparativa real con precios del momento.

1.2 Arquitectura Tecnica

```
                    +-------------------+
                    |   WhatsApp Cloud  |
                    |   API (Meta)      |
                    +--------+----------+
                             |
                    +--------v----------+
                    |   FastAPI Backend  |
                    |   (Python 3.11+)  |
                    +--------+----------+
                             |
          +------------------+------------------+
          |                  |                  |
   +-------v-------+  +------v------+  +--------v--------+
   |   SQLite DB   |  | Claude AI   |  |  Stripe Payments|
   |  (SQLAlchemy) |  | (Anthropic) |  |  (Checkout)     |
   +---------------+  +------+------+  +-----------------+
                             |
                    +--------v----------+
                    |   Groq Whisper    |
                    |   (Audio -> Text) |
                    +-------------------+
```

Stack principal:

- Backend: FastAPI (Python) con SQLAlchemy ORM
- Base de datos: SQLite (desarrollo), migracion a PostgreSQL en produccion
- Mensajeria: WhatsApp Cloud API (Meta) + Twilio como fallback
- IA: Anthropic Claude (interpretacion de pedidos, parseo de respuestas)
- Voz: Groq Whisper (transcripcion de notas de voz)
- Pagos: Stripe Checkout (tarjeta de credito/debito)
- Geocoding: Nominatim/OpenStreetMap (ubicacion GPS a direccion)
- Auth Web: Google OAuth (portales web)

1.3 Actores del Sistema

Actor	Descripcion	Canal principal
Cliente (Comprador)	Persona que necesita materiales de construccion	WhatsApp
Residente de obra	Personal en campo que pide materiales dia a dia	WhatsApp
Superintendente	Supervisa varios residentes, puede aprobar compra	WhatsApp
Compras	Departamento de adquisiciones de la constructora	WhatsApp / Web
Director	Maximo nivel de aprobacion corporativa	WhatsApp / Web
Proveedor	Empresa que vende materiales (ferreteria, cemente	WhatsApp / Web
Vendedor	Persona especifica dentro de un proveedor que atie	WhatsApp
Agente Nico (IA)	Agente inteligente que orquesta todo el flujo	Automatico

---

2. Proceso Principal: Pedido de Materiales

2.1 Inicio del Pedido (WhatsApp)

Entradas soportadas:

- Texto: "Necesito 15 metros cubicos de concreto fc 250 para manana en Zapopan"
- Audio: Nota de voz que se transcribe automaticamente con Groq Whisper
- Ubicacion: Pin de GPS que se convierte a direccion via reverse geocoding (Nominatim)

Flujo de deteccion de contexto:

Antes de procesar el mensaje, el sistema detecta en que estado se encuentra el usuario:

```
Mensaje entrante
      |
      v
  Es proveedor? ----SI----> manejar_respuesta_proveedor()
      |NO
      v
  Es APROBAR/RECHAZAR? --SI--> manejar_aprobacion() / manejar_rechazo()
      |NO
      v
  Detectar contexto del usuario:
      |
      +-- Pedido en "cotizando"    --> manejar_esperando_cotizaciones()
      +-- Pedido en "enviado"      --> manejar_seleccion_proveedor()
      +-- Orden en "en_obra"       --> manejar_confirmacion_entrega()
      +-- Orden activa (otra)      --> manejar_orden_activa()
      +-- Nada activo              --> manejar_nuevo_pedido()
```

Interpretacion por Claude AI:

El agente Claude analiza el mensaje del usuario y extrae:

- Items: producto, cantidad, unidad, categoria, especificaciones (resistencia, calibre, tipo)
- Entrega: direccion, municipio, fecha, horario
- Status: "completo" (tiene todo para cotizar) o "incompleto" (falta info)

Si el pedido esta incompleto, Nico responde pidiendo lo que falta (ej: "Me dices la direccion de entrega?").

2.2 Cotizacion Activa (Real-time)

Este es el corazon de ObraYa. NO es un catalogo estatico -- es cotizacion en vivo.

Paso 1: Seleccion de proveedores (hasta 20)

```
Criterios de seleccion (en orden de prioridad):
  1. Categorias: el proveedor maneja al menos 1 categoria del pedido
  2. Activo: proveedor habilitado en el sistema
  3. WhatsApp: tiene numero de WhatsApp registrado
  4. Geografia: bonus por cercania al punto de entrega
     - Mismo municipio: +5.0 puntos
     - Misma zona metropolitana: +3.0 puntos
     - Otra ciudad: -2.0 puntos
  5. Calificacion: mejor rating tiene prioridad
  6. Pedidos cumplidos: mas experiencia = mas arriba
  7. Tiempo de respuesta: mas rapido = mas arriba
```

Zonas metropolitanas soportadas:

- Guadalajara (Zapopan, Tlaquepaque, Tonala, Tlajomulco, El Salto, etc.)
- Monterrey (San Pedro, Apodaca, Guadalupe, Santa Catarina, etc.)
- CDMX (Naucalpan, Tlalnepantla, Ecatepec, etc.)
- Mazatlan, Culiacan, Leon, Queretaro, Puebla, Merida, Tijuana, Aguascalientes

Paso 2: Envio de mensaje al proveedor

```
Hola, buen dia. Soy Nico de ObraYa.

Tengo un cliente que necesita los siguientes materiales:

  - 15 m3 de concreto (fc 250)
  - 20 varillas de 3/8

Entrega en: Zapopan
Direccion: Av. Patria 1234

Me podrias pasar tu mejor precio con flete incluido?
Si puedes indicar tiempo de entrega, mejor.

Gracias!
```

Routing inteligente vendedor/proveedor:

```
Para cada proveedor:
  1. Buscar vendedor activo + disponible con menor tiempo de respuesta
  2. Si hay vendedor --> enviar a su WhatsApp personal
  3. Si no hay vendedor --> enviar al WhatsApp general del proveedor
```

El sistema intenta primero enviar un template de WhatsApp (funciona fuera de ventana 24h). Si el template no esta aprobado, envia texto libre como fallback.

Paso 3: Espera y recordatorios

```
Tiempo 0min  --> Solicitud enviada
Tiempo 15min --> Primer recordatorio a proveedores sin respuesta
Tiempo 25min --> Segundo recordatorio (maximo 2 recordatorios)
Tiempo 30min --> Timeout: se arma comparativa con lo que haya
```

Mensaje de recordatorio:

```
Hola, soy Nico de ObraYa.
Te habia mandado una solicitud de cotizacion hace rato:
[resumen del pedido]
```

```
Todavia tienes chance de cotizar?
Si no manejas algun material, no hay problema, dime.
```

Paso 4: Parseo de respuestas por IA

Cuando un proveedor responde (texto libre por WhatsApp), Claude AI parsea:

- tiene_precio: si incluye precio o no
- precio_total: monto total cotizado
- desglose: lista de items con precio unitario, cantidad, unidad, subtotal
- incluye_flete: si el precio incluye flete
- costo_flete: costo del flete por separado
- tiempo_entrega: "2 horas", "manana", etc.
- notas: condiciones adicionales
- sin_stock: si dijo que no tiene material

Paso 5: Registro de precios historicos

Cada respuesta de proveedor se registra automaticamente en la tabla PrecioHistorico:

- Producto normalizado contra catalogo maestro
- Precio unitario + flete prorrateado = precio efectivo
- Zona, fecha, dia de la semana, mes, trimestre
- Deteccion de outliers (precio > 3x o < 0.3x del promedio)

2.3 Comparativa de Precios

Cuando hay suficientes respuestas (minimo 1), se arma la comparativa:

```
Comparativa de precios para tu pedido #42:

  1. Materiales Zapopan -- $18,500
     - 15 m3 concreto fc250: $1,100/m3
     - Flete incluido
     - Entrega: 2 horas

  2. Concretos del Bajio -- $19,200
     - 15 m3 concreto fc250: $1,180/m3
     - Flete: $1,500
     - Entrega: manana 8am

Responde con el NUMERO del proveedor que prefieras (ej: 1)
```

El usuario responde "1", "el primero", "el mas barato", etc. El sistema interpreta respuestas naturales.

2.4 Cotizacion Estatica (Fallback)

Si no hay proveedores con WhatsApp o mientras se espera respuestas reales, se generan cotizaciones de la base de datos:

Motor de cotizacion (cotizador.py):

```
1. Usuario dice "varilla del tres octavos"
2. Buscar en aliases_producto --> catalogo_id = 6
3. Buscar todos los productos con catalogo_id = 6
4. Agrupar por proveedor, calcular subtotales
5. Sumar flete estimado por municipio
6. Ordenar por precio total
```

Calculo de flete por municipio (desde centro GDL):

Municipio	Flete estimado
Guadalajara	$800
Tlaquepaque	$1,000
Zapopan	$1,200
Tonala	$1,500
Tlajomulco	$1,800
El Salto	$2,000
Ixtlahuacan	$2,500

Reglas especiales:

- Mismo municipio proveedor/entrega: flete al 50%
- Proveedor grande con subtotal > $30,000: flete gratis

---

3. Proceso de Aprobacion Corporativa

3.1 Verificacion de Limites

```
Compra solicitada
      |
      v
  Usuario pertenece a empresa? --NO--> Compra directa (sin aprobacion)
```

```
        |SI
        v
    Empresa requiere aprobacion? --NO--> Compra directa
        |SI
        v
    Monto <= limite_sin_aprobacion de la empresa? --SI--> Compra directa
        |NO
        v
    El usuario puede auto-aprobar este monto? --SI--> Compra directa
        |NO
        v
    REQUIERE APROBACION --> solicitar_aprobacion()
```

Jerarquia de roles y limites (ejemplo demo):

Rol	Puede pedir	Puede aprobar	Puede pagar	Limite aprobacion
Residente	Si	No	No	$50,000
Superintendente	Si	Si	No	$150,000
Compras	Si	Si	Si	$500,000
Director	Si	Si	Si	Sin limite

3.2 Solicitud de Aprobacion

Al crearse la solicitud:

1. Se crea registro Aprobacion con status "pendiente" y expiracion en 24 horas
2. Se buscan todos los miembros de la empresa con puede_aprobar = True
3. Solo se notifica a aprobadores cuyo limite_aprobacion >= monto de la orden
4. Se envia WhatsApp a cada aprobador con contexto completo

Mensaje al aprobador:

```
  *Solicitud de Aprobacion*

  *Detalle del pedido:*
    - Concreto fc250 x15 m3 -- $16,500
    - Varilla 3/8 x20 pza -- $2,000
    *Total: $18,500 MXN*

  *Solicitante:* Juan Perez (residente)
  *Entrega:* Av. Patria 1234, Zapopan

  *Consumo acumulado:* $125,000 de $500,000 limite
```

```
*Alerta de presupuesto:*
La obra "Torre Norte" lleva 65% consumido
($325,000 de $500,000)
Este pedido llevaria el consumo a 69%

Para responder:
  APROBAR 42 -- autorizar compra
  RECHAZAR 42 [motivo] -- rechazar

Expira en 24 horas.
```

3.3 Aprobacion / Rechazo

Via WhatsApp:

- APROBAR 42 --> aprueba orden #42
- APROBAR 42 adelante con todo --> aprueba con nota
- RECHAZAR 42 muy caro, busca otra opcion --> rechaza con motivo

Via portal web: boton de aprobar/rechazar en /aprobaciones/

Validaciones al aprobar:

- El aprobador debe pertenecer a la misma empresa
- Debe tener puede_aprobar = True y estar activo
- Su limite_aprobacion debe ser >= al monto de la orden

Expiracion:

- Aprobaciones pendientes por mas de 24 horas se marcan como "expirada"
- Un scheduler periodico ejecuta verificar_expiradas()

Notificacion al solicitante:

```
(Aprobada)
*Compra aprobada!*
Orden #42 -- $18,500 MXN
Aprobada por: Carlos Gonzalez
Tu pedido se esta procesando.

(Rechazada)
*Compra rechazada*
Orden #42 -- $18,500 MXN
Rechazada por: Carlos Gonzalez
Motivo: muy caro, busca otra opcion
Si necesitas ajustar el pedido, mandame mensaje.
```

4. Proceso de Orden y Entrega

4.1 Creacion de Orden

Cuando el usuario elige un proveedor:

```
Usuario responde "1"
      |
      v
  Crear Orden desde Cotizacion:
    - proveedor_id
    - items (JSON)
    - total
    - direccion_entrega
    - status: "confirmada"
      |
      v
  Registrar en SeguimientoEntrega (timeline)
      |
      v
  Incrementar total_pedidos del proveedor
      |
      v
  Auto-consumo presupuestal:
    - Buscar presupuesto activo del usuario
    - Sumar al gastado_total
    - Recalcular porcentaje_consumido
      |
      v
  Necesita aprobacion? --SI--> Flujo de aprobacion (seccion 3)
      |NO
      v
  Notificar al usuario: "Pedido confirmado con [proveedor]!"
  Limpiar historial de conversacion
```

4.2 Ciclo de Vida de la Orden

```
  confirmada --> preparando --> en_transito --> en_obra --> entregada
                                                  |
                                                  +--> con_incidencia --> entregada


  (Cualquier estado) --> cancelada
```

Transiciones validas:

Desde	Hacia
confirmada	preparando, cancelada
preparando	en_transito, cancelada
en_transito	en_obra, cancelada
en_obra	entregada, con_incidencia, cancelada
con_incidencia	entregada, cancelada

Timestamps registrados por etapa:

- confirmada_at, preparando_at, en_transito_at, en_obra_at, entregada_at, cancelada_at

Datos de transporte (al pasar a en_transito):

- nombre_chofer
- telefono_chofer
- placas_vehiculo
- tipo_vehiculo

Al entregarse:

- Se calcula tiempo_entrega_minutos (desde en_transito hasta entregada)
- Se incrementa pedidos_cumplidos del proveedor
- Se registra fecha_entrega_real

4.3 Tracking de Entrega (SeguimientoEntrega)

Cada cambio de status se registra en la tabla SeguimientoEntrega:

- status_anterior
- status_nuevo
- origen (usuario, admin, proveedor, agente)
- nota (contexto del cambio)
- created_at (timestamp)

Esto genera un timeline completo consultable via API.

4.4 Agente Proactivo -- Alertas Automaticas

El agente proactivo se ejecuta cada 10-15 minutos y realiza las siguientes acciones:

1. Recordatorio de entrega proxima a proveedores

- Condicion: orden en "confirmada" o "preparando" con fecha de entrega para hoy o manana
- Mensaje al proveedor con datos de la orden
- Pide responder PREPARANDO o PROBLEMA

2. Alerta de recepcion a la obra
   - Condicion: orden cambio a "en_transito" en los ultimos 5 minutos
   - Mensaje al usuario con datos del chofer, placas, vehiculo
   - Pide responder RECIBIDO o PROBLEMA
3. Confirmacion de entrega (insistencia)
   - Condicion: orden en "en_obra" por mas de 2 horas sin confirmacion
   - Se envia a las 2h y 6h
   - No se insiste despues de 24 horas
4. Recordatorio de compromiso a proveedor
   - Condicion: orden en "confirmada" por mas de 4 horas sin avanzar a "preparando"
   - Se envia a las 4h y 12h
   - No se insiste despues de 48 horas
5. Solicitud de calificacion post-entrega
   - Condicion: orden entregada hace exactamente 24 horas
   - Solo si el usuario no ha calificado aun
   - Pide calificacion 1-5 estrellas
6. Alerta de retraso de entrega
   - Condicion: fecha_entrega_prometida ya paso y la orden no esta entregada
   - Se alerta a AMBAS partes (proveedor y comprador)
   - Se envia a la 1h, 4h y 12h de retraso
   - No se insiste despues de 72 horas
7. Recordatorio de cotizaciones pendientes
   - Condicion: solicitud a proveedor con 15+ minutos sin respuesta
   - Maximo 2 recordatorios por solicitud
   - Se envia a los 15 y 25 minutos

4.5 Incidencias

Tipos de incidencia:

Tipo	Descripcion	Severidad default
cantidad_incorrecta	Llego menos material del pedido	media

especificacion	Llego material diferente al pedido	grave
entrega_tarde	No llego a la hora acordada	leve
material_danado	Material roto, mojado, golpeado	media
no_llego	El pedido nunca llego	grave
cobro_diferente	El precio cobrado no coincide con la cotizacion	media
otro	Cualquier otro problema	media

Auto-clasificacion desde texto:

El sistema analiza el mensaje del usuario buscando palabras clave:

- "faltaron 5 metros" --> cantidad_incorrecta
- "no era lo que pedi" --> especificacion
- "llego 3 horas tarde" --> entrega_tarde
- "las varillas venian dobladas" --> material_danado
- "nunca llego" --> no_llego
- "me cobraron de mas" --> cobro_diferente

Intensificadores de severidad: "nunca", "jamas", "todo mal", "inaceptable", "pesimo" --> severidad "grave"

Extraccion inteligente de cantidades:

El sistema intenta extraer datos numericos del texto:

- "pedi 15 metros y llegaron 12" --> esperado: 15, recibido: 12, unidad: m3
- "20 bultos y solo vinieron 18" --> esperado: 20, recibido: 18, unidad: bultos

Flujo de resolucion:

```
Incidencia creada (status: "abierta")
      |
      v
  Orden pasa a status "con_incidencia"
      |
      v
  Notificacion al usuario confirmando registro
      |
      v
  Admin resuelve: resolver_incidencia(id, resolucion)
      |
      v
  Status: "resuelta" + fecha_resolucion
```

5. Sistema de Pagos (Stripe)

5.1 Desglose de Montos

```
Subtotal (precio del proveedor)     $18,500.00
+ Comision ObraYa (2%)              +   $370.00
----------------------------------------
Total cobrado al cliente            $18,870.00

Monto que recibe el proveedor:      $18,500.00
Comision ObraYa:                       $370.00
```

5.2 Flujo de Pago

```
1. Crear sesion Stripe Checkout
   - line_item: "ObraYa - Orden #42"
   - monto: total + 2% comision (en centavos MXN)
   - metadata: orden_id, subtotal, comision
   |
   v
2. Redirigir al cliente a Stripe (URL de checkout)
   |
   v
3. Cliente paga con tarjeta en Stripe
   |
   v
4. Stripe redirige a /pagos/exito?session_id=xxx
   |
   v
5. confirmar_pago():
   - Recuperar sesion de Stripe
   - Marcar orden como pagada
   - Registrar: metodo_pago, stripe_payment_id, fecha_pago
   - Registrar: comision_obraya, monto_proveedor
   - pago_proveedor_status = "pendiente"
   - Actualizar credit scoring del usuario
   |
   v
6. Pago al proveedor (manual/SPEI):
   - registrar_pago_proveedor()
   - pago_proveedor_status = "pagado"
```

5.3 Simulacion (Testing)

Endpoint simular_pago(orden_id) que:

- Marca la orden como pagada sin pasar por Stripe
- metodo_pago = "simulado"
- stripe_payment_id = "sim_{orden_id}_{timestamp}"
- Ejecuta la misma logica de credit scoring

---

6. Presupuestos de Obra

6.1 Creacion de Presupuesto

```
crear_presupuesto(
    nombre_obra="Torre Norte Fase 2",
    direccion="Av. Patria 1234, Zapopan",
    fecha_inicio=...,
    fecha_fin_estimada=...,
    partidas=[
        {
            "nombre_material": "Concreto fc 250",
            "categoria": "concreto",
            "unidad": "m3",
            "cantidad_presupuestada": 500,
            "precio_unitario_estimado": 1200,
            "catalogo_id": 1
        },
        ...
    ]
)
```

Cada partida tiene:

- Nombre del material y categoria
- Unidad de medida
- Cantidad presupuestada y precio unitario estimado
- Monto presupuestado (cantidad x precio)
- Campos de tracking: cantidad_consumida, monto_gastado, porcentaje_consumido
- Flags de alerta: alerta_50_enviada, alerta_80_enviada, alerta_100_enviada

- Flag de bloqueo: bloqueado (true/false)

6.2 Tracking de Consumo

Registro automatico al comprar:

Al crear una orden, el sistema automaticamente:

1. Busca el presupuesto activo del usuario
2. Suma el total de la orden al gastado_total
3. Recalcula porcentaje_consumido

Registro manual por partida:

registrar_consumo() actualiza una partida especifica:

- Incrementa cantidad_consumida y monto_gastado
- Recalcula porcentaje de la partida
- Recalcula totales del presupuesto
- Dispara verificacion de alertas

6.3 Alertas por WhatsApp

```
Consumo al 50%:
  "Tu partida de concreto para Torre Norte ya lleva el 50%
  consumido (250/500 m3). Vas bien, pero toma nota."

Consumo al 80%:
  "*ALERTA:* Tu partida de concreto ya va al 80%.
  Solo te quedan 100 m3 del presupuesto."

Consumo al 100%:
  "*PRESUPUESTO AGOTADO:* La partida de concreto llego al 100%.
  Se bloqueo la compra de este material.
  Contacta a tu administrador para ampliar el presupuesto."
```

Al 100% la partida se bloquea automaticamente. Se puede desbloquear manualmente por un admin.

6.4 Verificacion de Disponibilidad

Antes de cada compra, verificar_disponibilidad() revisa:

- Existe presupuesto activo con partida para ese material?

- La partida esta bloqueada?
- La cantidad solicitada excede lo restante?

Respuestas posibles:

- permitido: True --> la compra puede proceder
- permitido: False, partida bloqueada --> presupuesto agotado
- permitido: False, excede disponible --> la cantidad solicitada es mayor a lo restante
- Sin presupuesto --> compra libre (no hay restriccion)

---

7. Credit Scoring

7.1 Calculo del Score (0--100)

Componente	Peso	Que mide
Historial de pagos	40%	Ratio pagos a tiempo vs tarde (<=7 dias = a tiempo
Volumen de compras	20%	Total gastado, normalizado a $500k MXN = 100%
Frecuencia	15%	Pedidos completados, normalizado a 50 pedidos = 10
Antiguedad	10%	Dias desde registro, normalizado a 365 dias = 100%
Diversidad proveedores	10%	Proveedores distintos usados, normalizado a 10 = 1
Metodo de pago	5%	Tarjeta=100, Transferencia=70, Efectivo=40

Formula:

```
Score = (pagos * 0.40) + (volumen * 0.20) + (frecuencia * 0.15)
      + (antiguedad * 0.10) + (diversidad * 0.10) + (metodo * 0.05)
```

7.2 Clasificacion

Score	Nivel	Riesgo
85-100	Excelente	Bajo
70-84	Bueno	Bajo
50-69	Regular	Medio
0-49	Malo	Alto
Sin pedidos	Sin historial	N/A

7.3 Evaluacion de Credito

Requisitos minimos para credito:

- Al menos 5 pedidos completados
- Score >= 65

Limite de credito sugerido:

```
limite = score * total_gastado * 0.1 / 100
```

Ejemplo: score 80, $200,000 gastado --> limite = $16,000

Actualizacion automatica:

Cada vez que se confirma un pago:

1. Se actualiza total_gastado del usuario
2. Se incrementa total_pedidos_completados
3. Se calcula promedio movil de dias de pago
4. Se clasifica como pago a tiempo (<=7 dias) o tarde
5. Se recalcula el score completo

---

8. Calificacion de Proveedores

8.1 Calificacion Auto-calculada

Se ejecuta automaticamente al confirmar entrega de una orden.

Eje	Peso	Como se calcula
Puntualidad	35%	5.0 base, -1 punto por cada 30 min de retraso
Cantidad correcta	25%	5.0 base, se reduce por ratio recibido/esperado
Especificacion correcta	25%	5.0 si correcto, 2.0 si incidencia de especificaci
Sin incidencias	15%	5.0 base, -1.5 por cada incidencia registrada

Resultado: calificacion total ponderada de 1.0 a 5.0

8.2 Calificacion Manual

24 horas despues de la entrega, Nico pide calificacion al usuario:

```
Como calificas la experiencia? (1-5 estrellas)
1 - Muy mal
2 - Mal
3 - Regular
4 - Bien
5 - Excelente

Tambien puedes agregar un comentario despues del numero.
```

8.3 Impacto en el Sistema

Al registrar una calificacion se recalculan las metricas agregadas del proveedor (ultimas 50 ordenes):

- calificacion: promedio general (1.0-5.0)
- tasa_puntualidad: % de ordenes con puntualidad >= 4.0
- tasa_cantidad_correcta: % de ordenes con cantidad >= 4.5
- tasa_especificacion_correcta: % de ordenes con spec >= 4.5
- total_incidencias: conteo total de incidencias
- total_ordenes_completadas: conteo de ordenes entregadas

Estas metricas afectan directamente el orden de seleccion de proveedores en cotizacion activa.

---

9. Inteligencia de Precios

9.1 Registro de Precios

Cada cotizacion de proveedor se guarda automaticamente en PrecioHistorico:

Campo	Descripcion
catalogo_id	Producto normalizado del catalogo maestro
producto_nombre	Nombre original como lo dijo el proveedor
producto_normalizado	Nombre del catalogo maestro
proveedor_id/nombre	Quien cotizo
precio_unitario	Precio por unidad

precio_efectivo	Precio + flete prorrateado
unidad	m3, pieza, bulto, etc.
cantidad_cotizada	Volumen cotizado
incluye_flete	Si/No
costo_flete	Costo del flete (prorrateado)
zona	Zona geografica
tiempo_entrega	Compromiso de entrega
fuente	"cotizacion_activa", "manual", "referencia"
fecha, dia_semana, mes, anio, trimestre	Para analisis temporal
es_outlier	Flag de precio anomalo
confianza	Nivel de confianza del dato (0-1)

9.2 Deteccion de Outliers

Un precio se marca como outlier si:

- Hay al menos 5 datos historicos para ese producto+unidad
- El precio es > 3x del promedio historico, o < 0.3x del promedio

Los outliers se excluyen del calculo de precio de referencia.

9.3 Precio de Referencia del Catalogo

El catalogo maestro mantiene un precio_referencia que se recalcula automaticamente:

- Se toman los ultimos 30 precios no-outlier
- Se calcula el promedio
- Se actualiza en el catalogo

9.4 Analytics Disponibles

- Tendencia de precio: precio promedio/min/max por mes, ultimos 6-12 meses
- Variacion mensual: cambio % mes a mes, deteccion de tendencia (alza/baja/estable)
- Ranking de proveedores: por precio promedio efectivo para un producto
- Resumen de mercado: total de datos, proveedores y productos trackeados, top 5 productos mas cotizados
- Precio actual: promedio de ultimas 10 cotizaciones no-outlier

10. Gestion de Equipos

10.1 Empresas

Campo	Descripcion
nombre	Razon social
rfc	RFC de la empresa
direccion	Direccion fiscal
telefono	Telefono de contacto
email	Email corporativo
requiere_aprobacion	Si las compras necesitan autorizacion
limite_sin_aprobacion	Monto maximo para compra directa sin autorizacion
activo	Si la empresa esta habilitada

10.2 Miembros de Empresa

Cada miembro tiene:

Campo	Descripcion
rol	residente, superintendente, compras, director
puede_pedir	Puede solicitar materiales
puede_aprobar	Puede autorizar compras de otros
puede_pagar	Puede realizar pagos
limite_aprobacion	Monto maximo que puede aprobar (null = sin limite)
activo	Si el miembro esta habilitado

10.3 Vendedores (Personal de Proveedores)

Cada proveedor puede tener multiples vendedores:

Campo	Descripcion
nombre	Nombre del vendedor
telefono_whatsapp	Su WhatsApp personal
rol	vendedor, gerente, mostrador, almacen
solicitudes_atendidas	Contador de solicitudes

tiempo_respuesta_promedio	En minutos
calificacion	Rating individual (1-5)
disponible	True/False (vacaciones, etc.)
horario	"L-V 8:00-18:00"
categorias_especialidad	JSON con categorias que domina
activo	Si esta habilitado

Routing inteligente:

Al contactar un proveedor, el sistema busca el mejor vendedor:

1. Activo y disponible
2. Ordenado por tiempo_respuesta_promedio (mas rapido primero)
3. Si no hay vendedor disponible, usa el WhatsApp general del proveedor

---

11. Plataformas Web

11.1 Hub Central (`/hub/`)

Pagina principal con acceso a todos los modulos. Dashboard general del sistema.

11.2 Portal Clientes (`/portal/`)

Portal de autoservicio para compradores:

- Ver ordenes activas y completadas
- Timeline de seguimiento
- Historial de pedidos
- Calificaciones dadas

11.3 Portal Proveedores (`/portal/`)

Portal de autoservicio para proveedores:

- Solicitudes pendientes de cotizacion
- Ordenes en proceso
- Metricas de desempeno
- Calificaciones recibidas

11.4 Panel Admin (`/admin/`)

Panel de administracion del sistema:

- CRUD de proveedores, productos, usuarios
- Gestion de catalogo maestro
- Importacion de precios historicos
- Resolucion de incidencias

11.5 Dashboard Analytics (`/dashboard/`)

Metricas y graficas:

- Ordenes por status
- Ingresos y comisiones
- Proveedores mas usados
- Tiempos de respuesta

11.6 Simulador WhatsApp (`/sim/`)

Simulador web para probar el flujo de WhatsApp sin necesidad de un numero real. Permite:

- Enviar mensajes como comprador
- Enviar respuestas como proveedor
- Ver el flujo completo en tiempo real

11.7 Inteligencia de Precios (`/precios/`)

Panel de analisis de precios:

- Tendencias por producto
- Comparativa entre proveedores
- Deteccion de outliers
- Top productos mas cotizados

11.8 Presupuestos (`/presupuesto/`)

Gestion de presupuestos de obra:

- Crear/editar presupuestos con partidas
- Visualizar consumo por partida
- Alertas de porcentaje

11.9 Aprobaciones (`/aprobaciones/`)

Portal web para aprobar/rechazar ordenes:

- Lista de aprobaciones pendientes
- Detalle de cada solicitud
- Botones de aprobar/rechazar

11.10 Credit Scoring (`/credito/`)

Panel de credit scoring:

- Perfil crediticio de cada usuario
- Ranking de usuarios por score
- Evaluacion de elegibilidad

11.11 Pagos (`/pagos/`)

Modulo de pagos:

- Iniciar pago Stripe
- Simular pago (testing)
- Historial de pagos
- Pagos pendientes a proveedores

---

12. Integraciones Externas

Servicio	Uso	Configuracion
WhatsApp Cloud API (Meta)	Canal principal de comunicacion con usuarios y pro	WHATSAPP_TOKEN, PHONE_NUMBER_ID, VERIFY_TO
WhatsApp via Twilio	Fallback cuando Meta Cloud API no funciona	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN
Anthropic Claude	Interpretacion de pedidos, parseo de respuestas de	ANTHROPIC_API_KEY
Groq Whisper	Transcripcion de notas de voz a texto	GROQ_API_KEY
Stripe	Procesamiento de pagos con tarjeta, checkout	STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY
Nominatim/OpenStreetMap	Conversion de coordenadas GPS a direcciones leg	Sin API key (rate-limited)
Google OAuth	Autenticacion en portales web	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET

---

13. Cuentas Demo

Empresa Demo

Campo	Valor
Nombre	Constructora Demo SA de CV
RFC	CDM000101XXX
Direccion	Av. Patria 1234, Zapopan, Jalisco
Requiere aprobacion	Si
Limite sin aprobacion	$100,000 MXN

Usuarios Demo

Telefono	Nombre	Rol	Puede pedir	Puede aprobar	Puede pagar	Limite
5200000001	Admin Demo	Director	Si	Si	Si	Sin limite
5200000002	Residente Demo	Residente	Si	No	No	$50,000
5200000003	Superintendente Dem	Superintendente	Si	Si	No	$150,000
5200000004	Compras Demo	Compras	Si	Si	Si	$500,000

Proveedores Demo

Telefono WA	Nombre	Tipo	Municipio	Categorias	Calificacion
5200000010	Materiales Zapopan Dem	Mediano	Zapopan	acero, cemento, arena, g	4.2
5200000011	Ferreteria GDL Demo	Pequeno	Guadalajara	ferreteria, plomeria, elect	4.5
5200000012	Concretos del Bajio Dem	Grande	Tlaquepaque	concreto, cemento, morte	3.8

Como probar

1. Simulador web: Usar /sim/ para enviar mensajes como cualquier usuario demo
2. Flujo comprador: Enviar mensaje desde telefono 5200000002 (Residente) pidiendo material
3. Flujo proveedor: Responder desde telefono 5200000010 con precios
4. Flujo aprobacion: Crear orden > $100,000 desde Residente --> Director recibe solicitud
5. Pagos: Usar endpoint de simulacion para probar sin Stripe real

Anexo: Flujo Completo End-to-End

```
RESIDENTE                    NICO (IA)                   PROVEEDORES
    |                            |                            |
    |-- "Necesito 15m3 de       |                            |
    |   concreto fc250 para     |                            |
    |   manana en Zapopan" ---->|                            |
    |                            |                            |
    |                            |-- Interpretar con Claude   |
    |                            |-- Seleccionar 20 provs     |
    |                            |                            |
    |                            |-- "Hola, soy Nico..." --->|-- Proveedor A
    |                            |-- "Hola, soy Nico..." --->|-- Proveedor B
    |                            |-- "Hola, soy Nico..." --->|-- Proveedor C
    |                            |                            |
    |<-- "Estoy cotizando con   |                            |
    |    3 proveedores..."      |                            |
    |                            |                            |
    |                            |<-- "$16,500 con flete" ----|-- Proveedor A
    |<-- "1 proveedor respondio"|                            |
    |                            |                            |
    |                            |<-- "$18,200 sin flete" ----|-- Proveedor C
    |                            |                            |
    |                            |-- (15min sin resp.) ------>|-- Recordatorio a B
    |                            |                            |
    |<-- "Comparativa:          |                            |
    |    1. Zapopan $16,500     |                            |
    |    2. Bajio $19,700       |                            |
    |    Elige un numero" ------|                            |
    |                            |                            |
    |-- "1" ------------------->|                            |
    |                            |                            |
    |                            |-- Verificar aprobacion     |
    |                            |-- Crear orden              |
    |                            |-- Registrar presupuesto    |
    |                            |                            |
    |<-- "Pedido confirmado     |                            |
    |    con Materiales Zapopan |                            |
    |    Orden #42, $16,500" ---|                            |
    |                            |                            |
    |                   ... HORAS DESPUES ...                |
    |                            |                            |
    |                            |-- Alerta entrega -------->|-- Proveedor A
    |<-- "Tu pedido va en       |                            |
    |    camino! Chofer: Juan   |                            |
```

```
|    Placas: ABC-123" ------|                            |
|                           |                            |
|<-- "El camion ya llego.   |                            |
|    Responde OK o          |                            |
|    Problema" -------------|                            |
|                           |                            |
|-- "OK" ------------------>|                            |
|                           |                            |
|                           |-- Calcular calificacion    |
|                           |-- Actualizar metricas      |
|                           |-- Registrar precios hist.  |
|                           |                            |
|<-- "Entrega completada!   |                            |
|    Calificacion: 4.7/5    |                            |
|    Gracias por usar       |                            |
|    ObraYa!" --------------|                            |
|                           |                            |
|                   ... 24 HORAS DESPUES ...             |
|                           |                            |
|<-- "Como calificas tu     |                            |
|    pedido? (1-5)" --------|                            |
|                           |                            |
|-- "5 excelente servicio"->|                            |
|                           |-- Registrar calificacion   |
|                           |-- Recalcular metricas prov |
```

Documento generado como referencia operativa del sistema ObraYa.

Ultima actualizacion: Abril 2026

PARTE 2

Analisis Exhaustivo de Variantes

ANALISIS EXHAUSTIVO DE VARIANTES Y EDGE CASES — ObraYa

Fecha: 2026-04-12

Objetivo: Identificar TODAS las variantes, problemas, fallas y edge cases que pueden ocurrir en produccion, con estrategias concretas para cada uno.

PROCESO 1: PEDIDO DE MATERIALES (Nuevo Pedido)

1.1 Mensaje vago del usuario

Escenario: El usuario manda "necesito material" o "necesito cemento" sin cantidad, direccion ni fecha.

Estado actual: Claude (agente_claude.py) interpreta el mensaje y devuelve status="incompleto" con un mensaje pidiendo mas datos. El sistema envia ese mensaje al usuario y espera otra respuesta.

Problemas detectados:

- No hay limite de cuantas veces puede ir y venir la conversacion incompleta. Un usuario podria mandar 15 mensajes vagos y nunca completar el pedido.
- No hay timeout de conversacion incompleta. Si el usuario manda "necesito material" y no responde en 3 horas, la conversacion queda abierta en Redis/memoria de Claude sin limpiar.
- El historial de conversacion crece indefinidamente si el usuario manda muchos mensajes antes de completar.

Estrategias propuestas:

- Limite de turnos: Despues de 5 intercambios sin completar, enviar: "Parece que necesitas ayuda mas personalizada. Mandame un audio con todos los detalles de tu pedido y yo lo proceso."
- Timeout de conversacion: Si pasan 2 horas sin actividad en una conversacion incompleta, limpiar el historial y la proxima vez empezar de cero. Mandar mensaje: "Tu solicitud anterior expiro. Mandame de nuevo tu pedido cuando estes listo."
- Resumen proactivo: Despues del 3er intercambio, Claude deberia resumir lo que ya tiene y solo preguntar lo que falta: "Ya tengo: 50 bultos de cemento. Me falta: direccion de entrega y fecha."

1.2 Falla en transcripcion de audio

Escenario: El usuario manda una nota de voz pero la transcripcion falla (ruido de obra, audio muy corto, formato incompatible).

Estado actual: El webhook maneja dos casos: (a) no se pudo descargar el audio, y (b) no se pudo transcribir. En ambos pide al usuario que reintente o escriba.

Problemas detectados:

- No hay reintentos automaticos. Si Whisper falla por un error transitorio, el usuario tiene que volver a grabar.
- No se guarda el audio original para reintento manual o debug.
- Un audio muy largo (5+ minutos) podria exceder limites de la API de transcripcion.
- Si el usuario esta en una zona con senal intermitente, el audio podria llegar incompleto.

Estrategias propuestas:

- Reintento automatico: Si la transcripcion falla una vez, reintentar con parametros diferentes (idioma forzado a "es", temperature=0).
- Guardar audio en S3/bucket: Para auditoria y reintento manual. Si un pedido grande falla, un humano puede escucharlo.
- Limite de duracion: Si el audio pasa de 3 minutos, avisar al usuario: "Tu audio es muy largo. Intentare procesarlo, pero si falla, mandame los datos clave por texto."
- Fallback a texto: Ofrecer una plantilla: "Mandame tu pedido asi: MATERIAL: cemento / CANTIDAD: 50 bultos / ENTREGA: Zapopan / FECHA: manana"

1.3 Usuario envia imagen

Escenario: El usuario manda una foto de una lista de materiales, un plano, o una foto del material que necesita.

Estado actual: El webhook responde "Por ahora puedo recibir texto, audio y ubicacion" y descarta la imagen.

Problemas detectados:

- Perder un pedido porque el residente de obra mando una foto de su lista es un caso de uso MUY comun en construccion. Los residentes suelen tener listas impresas o en papel que fotografian.
- No hay vision/OCR integrado.
- El usuario podria frustrarse y no volver a intentar.

Estrategias propuestas:

- Corto plazo: Responder con un mensaje mas util: "Todavia no puedo leer imagenes, pero pronto. Por ahora, dicta tu pedido por audio o escribelo. Tip: manda un audio leyendo la lista de materiales."
- Mediano plazo: Integrar GPT-4V o Claude Vision para interpretar imagenes de listas de materiales. Extraer texto con OCR y pasarlo al flujo normal.
- Largo plazo: Soportar PDFs (planos, presupuestos) y extraer la lista de materiales automaticamente.

1.4 Material no existe en catalogo

Escenario: El usuario pide "geotextil" o "membrana impermeabilizante" pero no hay ningun proveedor que lo maneje en la BD.

Estado actual: generar_cotizaciones retorna lista vacia. El sistema dice "No encontre proveedores disponibles para esos materiales. Intenta con otros o contactanos directo." y cancela el pedido.

Problemas detectados:

- Se pierde el pedido sin ningun follow-up. El usuario necesita ese material urgente y lo mandamos a "contactanos directo" sin dar un contacto real.
- No se registra la demanda insatisfecha. Si 50 usuarios piden geotextil y no tenemos, no sabemos que deberiamos agregar proveedores de geotextil.
- El cotizador usa FLETE_POR_MUNICIPIO hardcodeado solo para GDL. Cualquier ciudad fuera de GDL no tiene estimado de flete.

Estrategias propuestas:

- Registro de demanda insatisfecha: Crear tabla demanda_no_cubierta con: material solicitado, ciudad, fecha, usuario_id. Esto alimenta la estrategia de expansion de proveedores.
- Alerta a admin: Si un material se pide 3+ veces en una semana y no hay proveedor, enviar WhatsApp al admin: "Hay demanda de geotextil (5 solicitudes esta semana). Necesitamos agregar proveedores."
- Respuesta mejorada: En lugar de cancelar, ofrecer: "No tengo proveedores de geotextil aun, pero puedo buscarte uno manualmente. Te contesto en 1 hora." Y crear una tarea para el admin.
- Sugerencia de alternativas: Si el material es similar a otro del catalogo, sugerir: "No encontre 'geomembrana', pero tengo proveedores de 'membrana impermeabilizante'. Te sirve?"

1.5 Ciudad sin proveedores

Escenario: El usuario esta en Oaxaca pero solo hay proveedores registrados en Guadalajara, Monterrey y CDMX.

Estado actual: seleccionar_proveedores filtra por categoria y zona metropolitana. Si el municipio de entrega no esta en ninguna zona definida, los proveedores fuera de zona reciben penalizacion de -2.0 en score, pero aun asi se les contacta si tienen la categoria.

Problemas detectados:

- Se podrian contactar proveedores de Guadalajara para una entrega en Oaxaca. El proveedor diria "no entrego alla" y se pierde tiempo.
- ZONAS_METROPOLITANAS solo tiene 11 zonas. Mexico tiene 74 zonas metropolitanas. Faltan muchisimas ciudades.
- No hay validacion de que el proveedor REALMENTE entregue en esa zona. El campo municipio del proveedor no indica su cobertura real.

Estrategias propuestas:

- Campo de cobertura: Agregar a cada proveedor un campo cobertura_municipios (JSON array) que indique a que ciudades entrega. Filtrar solicitudes por cobertura real.
- Expandir zonas metropolitanas: Agregar al menos las 30 principales zonas de Mexico.
- Respuesta honesta: Si no hay proveedores en la zona: "No tengo proveedores activos en Oaxaca todavia. Estoy expandiendome. Te puedo avisar cuando tenga proveedores en tu zona."
- Registro de demanda geografica: Trackear en que ciudades se piden materiales para priorizar expansion.

1.6 Mensajes multiples en rapida sucesion

Escenario: El usuario manda 3 mensajes en 10 segundos:

1. "Necesito cemento"
2. "50 bultos"
3. "Para manana en Zapopan"

Estado actual: CADA mensaje se procesa de forma independiente en background_tasks.add_task(procesar_mensaje, msg, db). Esto significa que se crearian 3 flujos separados, cada uno incompleto.

Problemas detectados:

- Race condition grave: 3 tareas background corriendo en paralelo para el mismo usuario. Pueden crear 3 pedidos separados o interferir con la deteccion de contexto.
- La sesion de SQLAlchemy se comparte por request, no por background task. Las tareas background podrian tener problemas de concurrencia en la BD.
- No hay debouncing: no se espera un momento para acumular mensajes antes de procesarlos.

Estrategias propuestas:

- Debouncing: Cuando llega un mensaje, esperar 5 segundos antes de procesar. Si llegan mas mensajes en ese periodo, concatenarlos y procesarlos juntos.
- Lock por usuario: Usar un lock (Redis o asyncio.Lock por telefono) para que solo se procese un mensaje a la vez por usuario. Los demas esperan en cola.
- Concatenacion inteligente: Si el ultimo mensaje tiene menos de 30 segundos, concatenar con el anterior antes de enviarlo a Claude.
- Session por tarea: Crear una nueva sesion de SQLAlchemy dentro de cada background task para evitar race conditions en la BD.

1.7 Correccion mid-flow

Escenario: El usuario dijo "15 metros de concreto" pero inmediatamente despues dice "no espera, son 20 metros no 15".

Estado actual: Depende del timing. Si el primer mensaje ya completo el pedido y se creo, el segundo mensaje se

procesaria como un nuevo contexto (podria caer en "esperando_cotizaciones" o "seleccion_proveedor"). Si aun esta en la conversacion con Claude, Claude deberia interpretar la correccion.

Problemas detectados:

- Si el pedido ya se envio a proveedores (status "cotizando"), no hay forma de corregirlo. Los proveedores ya estan cotizando 15 metros, no 20.
- No hay comando de "cancelar" formalmente documentado para el usuario durante la fase de cotizacion.
- La logica de manejar_esperando_cotizaciones solo informa el progreso, no permite modificaciones.

Estrategias propuestas:

- Comando cancelar: Si el usuario escribe "cancelar" mientras esta en "cotizando", cancelar el pedido y permitir crear uno nuevo inmediatamente. (Ya existe un check para "cancelar" en manejar_esperando_cotizaciones en el texto que se muestra al usuario, pero NO esta implementado en el handler.)
- Edicion pre-envio: Si el usuario corrige ANTES de que se complete el pedido (status "incompleto"), Claude deberia usar el historial de conversacion para entender la correccion. Esto ya deberia funcionar si el historial se maneja bien.
- Re-cotizacion: Si ya se envio a proveedores, permitir cancelar las solicitudes pendientes y re-enviar con los datos corregidos. Avisar a los proveedores: "Disculpa, hubo un cambio en la solicitud."

1.8 Geocoding API caida

Escenario: El usuario envia un pin de ubicacion pero Nominatim (OpenStreetMap) no responde.

Estado actual: _reverse_geocode tiene un timeout de 10 segundos y un try/except. Si falla, retorna string vacio. El sistema aun tiene las coordenadas GPS y las usa como "Lat: X, Lng: Y".

Problemas detectados:

- Las coordenadas GPS crudas no le dicen nada al proveedor. "Entrega en: (20.6597, -103.3496)" no es util.
- No hay retry ni fallback a otro servicio de geocoding.
- No se cachean resultados de geocoding para reducir llamadas.

Estrategias propuestas:

- Retry con backoff: Reintentar 2 veces con 2 y 5 segundos de espera.
- Fallback a Google Maps API: Si Nominatim falla, usar Google Maps Geocoding como backup (requiere API key pero es mas confiable).
- Cache de geocoding: Redondear coordenadas a 3 decimales (~100m precision) y cachear en Redis. La mayoria de obras estan en la misma ubicacion por meses.
- Pedir direccion manual: Si el geocoding falla, preguntar: "Recibi tu ubicacion pero no pude obtener la direccion. Me la puedes escribir? (ej: Av. Americas 1500, Zapopan)"

1.9 Claude malinterpreta el pedido

Escenario: El usuario dice "block" y Claude interpreta "bloque de concreto" cuando queria "tabique rojo" (en algunas regiones de Mexico, "block" se usa para ambos).

Estado actual: No hay paso de confirmacion. Claude interpreta y si el pedido esta "completo", se envian solicitudes a proveedores directamente.

Problemas detectados:

- No hay paso de verificacion donde el usuario confirme que la interpretacion es correcta.
- Los regionalismos mexicanos son complejos: "piedra" puede ser piedra bola, piedra braza, grava, etc.
- Si Claude interpreta mal, se contactan proveedores por un material equivocado, se pierde tiempo de todos.

Estrategias propuestas:

- Confirmacion pre-cotizacion: Despues de interpretar, mostrar al usuario un resumen y pedir confirmacion: "Entendi tu pedido asi: 500 piezas de block de concreto 12x20x40 / Entrega en Zapopan manana / Correcto? Responde SI o corrigeme."
- Diccionario de regionalismos: Alimentar a Claude con un diccionario de regionalismos por estado/ciudad. "block" en Jalisco = bloque de concreto, "block" en Puebla = tabique, etc.
- Preguntas de desambiguacion: Si el producto tiene multiples interpretaciones posibles, Claude deberia preguntar: "Cuando dices 'block', te refieres a bloque de concreto (gris, hueco) o a tabique rojo?"

1.10 Pedido en ingles u otro idioma

Escenario: Un usuario extranjero o bilingue manda "I need 50 bags of cement delivered to Zapopan tomorrow".

Estado actual: Claude es multilingue, asi que probablemente interpretaria correctamente. Pero los mensajes del sistema estan todos en espanol.

Problemas detectados:

- La respuesta del sistema siempre sera en espanol, lo cual podria confundir a un usuario anglofono.
- Los proveedores reciben solicitudes en espanol, asi que la traduccion del pedido no es un problema para ellos.
- Los keywords de deteccion de contexto (es_confirmacion, es_reporte_problema, etc.) solo tienen palabras en espanol.

Estrategias propuestas:

- Deteccion de idioma: Si Claude detecta que el mensaje esta en ingles, responder en ingles pero aclarar que las comunicaciones con proveedores seran en espanol.
- Agregar keywords en ingles: Agregar "yes", "ok", "confirmed", "problem", "wrong" a las listas de deteccion.
- Campo de idioma en usuario: Guardar el idioma preferido del usuario para futuras comunicaciones.

PROCESO 2: COTIZACION ACTIVA

2.1 TODOS los proveedores offline / no responden

Escenario: Se contactan 20 proveedores y despues de 30 minutos ninguno ha respondido.

Estado actual: TIMEOUT_RESPUESTA_MINUTOS = 30. hay_suficientes_respuestas requiere MINIMO_RESPUESTAS_PARA_COMPARATIVA = 1. El agente proactivo envia recordatorios a los 15 y 25 minutos. Pero NO HAY un mecanismo que se ejecute al llegar a los 30 minutos de timeout para tomar accion.

Problemas detectados:

- CRITICO: No hay un scheduler/cron que ejecute tiempo_agotado() y tome accion cuando pasan los 30 minutos. La funcion existe pero nunca se llama automaticamente.
- marcar_sin_respuesta existe pero no se invoca de forma proactiva.
- El usuario queda en status "cotizando" indefinidamente si ningun proveedor responde y el agente proactivo no cubre este caso explicitamente.
- El agente proactivo (recordar_cotizaciones_pendientes) solo recuerda a los proveedores, no maneja el timeout del pedido.

Estrategias propuestas:

- URGENTE - Implementar timeout handler: En el agente proactivo, agregar una funcion que revise pedidos en status "cotizando" que llevan mas de 30 minutos. Si tiene al menos 1 respuesta, armar comparativa. Si tiene 0, notificar al usuario: "Ninguno de los proveedores que contactamos respondio. Opciones: 1) Esperar un poco mas, 2) Usar precios de referencia de nuestra BD, 3) Cancelar."
- Escalacion a admin: Si despues de 1 hora sigue sin respuestas, alertar al admin con detalles del pedido.
- Precios de referencia como fallback: Ya se generan precios de la BD; si no hay respuestas activas, usar esos como la comparativa principal (ya se hace parcialmente).

2.2 Solo 1 proveedor responde con precio altisimo

Escenario: De 20 proveedores contactados, solo 1 responde y su precio es $95,000 cuando el mercado esta en $60,000.

Estado actual: MINIMO_RESPUESTAS_PARA_COMPARATIVA = 1, asi que con 1 respuesta ya se arma la comparativa. No hay validacion de precio vs mercado.

Problemas detectados:

- Se le presenta al usuario una sola opcion con precio potencialmente inflado.
- No hay comparacion con precios historicos para detectar outliers.
- El usuario podria aceptar un precio abusivo por urgencia.

Estrategias propuestas:

- Comparacion con precios historicos: Si solo hay 1 respuesta, comparar con precio_historico de la BD. Si el precio es >30% arriba del promedio historico, advertir: "Este precio esta un 35% arriba del promedio del mercado ($X). Te sugerimos esperar a que respondan mas proveedores."
- Esperar mas tiempo con 1 respuesta: Si solo hay 1 respuesta al timeout de 30min, extender el timeout a 60min y enviar otro recordatorio.
- Mostrar precio de referencia: Junto con la unica cotizacion real, mostrar: "Precio de referencia del mercado: $60,000. Cotizacion recibida: $95,000."

2.3 Proveedor responde "dejame checarlo y te confirmo"

Escenario: El proveedor no da precio, solo dice que va a revisar.

Estado actual: parsear_respuesta_proveedor usa Claude para parsear. Claude deberia detectar que no hay precio y retornar tiene_precio: false. La solicitud se marca como "respondida" con precio_total = None.

Problemas detectados:

- Se marca como "respondida" sin precio, lo que podria confundir el conteo de respuestas. hay_suficientes_respuestas cuenta todas las "respondidas" sin verificar si tienen precio.
- El proveedor podria mandar el precio real 2 horas despues, pero la solicitud ya esta marcada como "respondida" y no se procesaria.
- No hay seguimiento automatico para pedirle el precio definitivo.

Estrategias propuestas:

- Status intermedio: Crear status "en_revision" para cuando el proveedor indica que va a revisar pero no da precio. No contar como respuesta para la comparativa.
- Solo contar respuestas con precio: Modificar hay_suficientes_respuestas para contar solo solicitudes donde precio_total IS NOT NULL.
- Recordatorio al proveedor: 15 minutos despues de "dejame checarlo", enviar: "Te recuerdo la cotizacion que quedaste de revisar. El cliente sigue esperando."
- Reabrir solicitud: Si el proveedor manda un segundo mensaje con precio, buscar la solicitud "respondida" sin precio y actualizarla.

2.4 Proveedor envia cotizacion como imagen/PDF

Escenario: El proveedor manda una foto de su nota o un PDF con precios.

Estado actual: El webhook solo procesa texto y audio. Si el proveedor manda una imagen, el mensaje se descarta (el tipo no es "texto", "audio", ni "ubicacion").

Problemas detectados:

- CRITICO para proveedores: Muchos proveedores en Mexico mandan cotizaciones como fotos de WhatsApp (foto de la hoja de precios, captura de pantalla de su sistema).
- Se pierde la cotizacion completamente.
- El proveedor no recibe feedback de que no pudimos leer su imagen.

Estrategias propuestas:

- Corto plazo: Detectar cuando un proveedor manda imagen y responder: "Gracias! Por ahora no puedo leer imagenes. Me puedes mandar los precios por texto? Ejemplo: 'Cemento $195 el bulto, flete $800. Entrego manana.'"
- Mediano plazo: Integrar OCR/Vision para leer cotizaciones de proveedores. Esto es prioritario porque la mayoria de proveedores estan acostumbrados a mandar fotos.
- Largo plazo: Aceptar PDFs, hojas de Excel, y fotos, y parsearlos automaticamente.

2.5 Proveedor cotiza items diferentes

Escenario: Se pidio "cemento CPC 40R" y el proveedor cotiza "cemento CPC 30R" porque no tiene el 40R.

Estado actual: Claude parsea la respuesta y extrae lo que hay. No valida que los items cotizados coincidan con los solicitados.

Problemas detectados:

- La comparativa podria mezclar manzanas con naranjas: un proveedor cotizo CPC 40R y otro CPC 30R.
- El usuario podria elegir sin darse cuenta de que no es el material exacto.
- No hay notas de "este proveedor ofrece un producto equivalente/diferente".

Estrategias propuestas:

- Validacion de items: Despues de parsear, comparar los productos cotizados con los solicitados. Si hay discrepancia, marcar en la comparativa: "OJO: Este proveedor cotizo CPC 30R en lugar de CPC 40R."
- Clasificacion de equivalencias: Usar el catalogo maestro para detectar si los productos son equivalentes (misma familia, diferente especificacion).
- Preguntar al proveedor: Si cotizo algo diferente, responder: "Gracias. Noto que cotizaste CPC 30R pero el cliente pidio CPC 40R. Tienes el 40R? Si no, registro tu cotizacion del 30R como alternativa."

2.6 Precio "sujeto a cambio"

Escenario: El proveedor dice "te lo dejo a 195 pero el precio puede cambiar, esta subiendo el cemento".

Estado actual: Claude parsearia el precio como 195 y pondria la nota en el campo "condiciones" o "notas".

Problemas detectados:

- El usuario podria aceptar la cotizacion y luego el proveedor dice "ya subio a 210". No hay proteccion contractual.
- El campo de vigencia de la cotizacion se calcula automaticamente (3 dias) pero el proveedor no se comprometio a eso.

Estrategias propuestas:

- Vigencia explicita: Preguntar al proveedor: "Perfecto, este precio de $195 es valido hasta cuando?"
- Advertencia al usuario: En la comparativa, marcar claramente: "Precio sujeto a cambio" junto a esa opcion.
- Confirmacion de precio: Al momento de crear la orden, enviar al proveedor: "El cliente acepto tu cotizacion de $X. Confirmas el precio para proceder?" No crear la orden hasta que el proveedor confirme.

2.7 Precios identicos de dos proveedores

Escenario: Proveedor A y Proveedor B dan exactamente el mismo precio.

Estado actual: La comparativa los muestra ordenados por precio. Si son iguales, el orden dependeria del orden de insercion en la BD.

Problemas detectados:

- No hay criterio de desempate visible para el usuario.
- El usuario no sabe por que elegir uno sobre otro.

Estrategias propuestas:

- Criterios de desempate en la comparativa: Si los precios son iguales, mostrar informacion diferenciadora: calificacion del proveedor, tiempo de respuesta promedio, pedidos cumplidos, si incluye flete.
- Recomendacion: Agregar un icono/texto "Recomendado" al proveedor con mejor calificacion historica.

2.8 Proveedor responde DESPUES del timeout

Escenario: Un proveedor responde con un buen precio 2 horas despues de que se armo la comparativa.

Estado actual: En manejar_respuesta_proveedor, se busca la solicitud pendiente con status "enviada" o "recordatorio_enviado". Si ya se marco como "sin_respuesta", no se encontraria y el proveedor recibiria: "Gracias por tu mensaje. Por el momento no tengo solicitudes pendientes para ti."

Problemas detectados:

- Se pierde una cotizacion potencialmente buena.
- El proveedor queda con mala impresion: le pedimos cotizacion y luego le decimos que no tenemos nada pendiente.
- Si el pedido aun no tiene orden creada (usuario no ha elegido), la cotizacion tardia podria ser util.

Estrategias propuestas:

- Aceptar respuestas tardias: Si el pedido esta en status "enviado" (comparativa ya mostrada, usuario aun no elige), aceptar la cotizacion tardia, actualizar la comparativa y re-enviar al usuario: "Un proveedor mas respondio. Tu comparativa actualizada:"
- Registrar para historico: Aunque sea tarde para este pedido, registrar el precio en el historico para futuros pedidos.
- Mensaje al proveedor: "Gracias por tu cotizacion. Para este pedido ya tenemos comparativa, pero registre tu precio

para futuras solicitudes. Si quieres que te incluyamos mas rapido, intenta responder en menos de 30 minutos."

2.9 Vendedor cambio de numero de WhatsApp

Escenario: El numero del vendedor que tenemos en la BD ya no es valido. WhatsApp marca el mensaje como no entregado.

Estado actual: Si enviar_mensaje_texto falla, la solicitud se marca como "error_envio". Pero no hay verificacion proactiva de numeros.

Problemas detectados:

- No se detecta automaticamente que un numero ya no existe.
- La solicitud queda como "error_envio" pero nadie hace nada al respecto.
- Si un proveedor tiene 3 vendedores y solo 1 cambio de numero, solo se falla ese vendedor, no los otros.

Estrategias propuestas:

- Detectar error de numero invalido: Analizar el error de WhatsApp API. Si el error es "number not registered", marcar el vendedor como activo = False.
- Fallback al numero principal del proveedor: Si el vendedor falla, intentar con el telefono_whatsapp del proveedor directamente.
- Alerta a admin: Si un proveedor tiene TODOS sus numeros fallando, alertar: "Proveedor X tiene todos sus numeros invalidos. Verificar manualmente."
- Verificacion periodica: Una vez al mes, enviar un "ping" a todos los numeros registrados para verificar que siguen activos.

2.10 Rate limiting de WhatsApp

Escenario: Se procesan 5 pedidos simultaneos y se necesita mandar 100 mensajes a proveedores en pocos segundos.

Estado actual: Los mensajes se envian secuencialmente dentro de enviar_solicitudes_a_proveedores (for loop). No hay control de rate.

Problemas detectados:

- WhatsApp Business API tiene limites: ~80 mensajes/segundo para nuevas conversaciones, pero solo despues de alcanzar cierto tier.
- Cuentas nuevas tienen limite de 250 conversaciones/24h, luego 1000, luego 10,000.
- Si se excede el rate, mensajes silenciosamente fallan o se encolan.

Estrategias propuestas:

- Rate limiter: Implementar un rate limiter global para la API de WhatsApp. Max 10 mensajes por segundo con cola de espera.

• Templates vs texto libre: Para nuevas conversaciones (fuera de ventana 24h), SOLO se pueden usar templates aprobados. Ya hay fallback a texto libre, pero este fallara si no hay ventana abierta.
• Monitoreo de tier: Trackear cuantas conversaciones se abren por dia. Alertar al admin cuando se acerque al 80% del limite del tier.
• Envio en batches: En lugar de mandar 20 mensajes de golpe, enviar en batches de 5 con 2 segundos entre cada batch.

2.11 Proveedor bloquea nuestro numero

Escenario: Un proveedor se harta de recibir solicitudes y bloquea el numero de Nico/ObraYa.

Estado actual: Los mensajes fallarian silenciosamente. WhatsApp no siempre informa explicitamente que fuimos bloqueados.

Problemas detectados:

• El mensaje se enviaria pero nunca llegaria. La solicitud quedaria en "enviada" para siempre.
• No se puede distinguir "bloqueado" de "no leido".
• Si un proveedor clave nos bloquea, podriamos perder cotizaciones buenas sin saberlo.

Estrategias propuestas:

• Deteccion por patron: Si un proveedor tiene 5+ solicitudes consecutivas sin respuesta (nunca responde), marcarlo como "posiblemente bloqueado" y dejar de enviarle.
• Verificar read receipts: Si WhatsApp informa que el mensaje fue "sent" pero nunca "delivered" despues de 30 minutos, es probable bloqueo.
• Canal alternativo: Para proveedores importantes, ofrecer contacto por telefono o email como alternativa.
• Feedback loop: Si un proveedor dice "dejen de mandarme mensajes", marcarlo como activo = False y respetar su decision.

2.12 Precio en USD en lugar de MXN

Escenario: Un proveedor cotiza "the unit is at 15 USD" o "son 15 dolares".

Estado actual: Claude parsearia el precio como 15 y lo registraria sin moneda. Se compararia con precios en MXN, mostrando una comparativa absurda.

Problemas detectados:

• No hay deteccion de moneda en el parser.
• Un precio de $15 USD apareceria como $15 MXN en la comparativa, pareciendo increiblemente barato.
• No hay conversion automatica de monedas.

Estrategias propuestas:

• Deteccion de moneda en parser: Agregar al prompt de Claude: "Si el proveedor menciona dolares, USD, o cualquier

moneda diferente a MXN, indicarlo en un campo 'moneda': 'USD'."

• Conversion automatica: Si se detecta USD, convertir a MXN con tipo de cambio del dia (API de Banxico o similar). Marcar en la comparativa: "Precio original: $15 USD (convertido a $270 MXN aprox)."

• Confirmar con proveedor: Si se detecta otra moneda: "Tu cotizacion fue en dolares, correcto? El equivalente en pesos es ~$270 MXN."

PROCESO 3: SELECCION DE PROVEEDOR

3.1 Usuario no responde a la comparativa (abandono)

Escenario: Se le manda la comparativa con 3 opciones y el usuario nunca responde.

Estado actual: El pedido queda en status "enviado" indefinidamente. detectar_contexto siempre lo encontrara como "seleccion_proveedor". Si el usuario manda cualquier mensaje futuro, se le pedira que elija proveedor.

Problemas detectados:

• El usuario queda "atrapado" en el flujo de seleccion. Si manda un mensaje 3 dias despues sobre otro tema, el sistema insiste en la seleccion.

• Hay un escape ("nuevo pedido", "otra cotizacion") pero no es obvio para el usuario.

• Las cotizaciones tienen vigencia de 3 dias, pero no se verifica la vigencia al momento de crear la orden.

Estrategias propuestas:

• Timeout de seleccion: Si el usuario no responde en 24 horas, enviar recordatorio: "Aun no has elegido proveedor para tu pedido de cemento. Las cotizaciones vencen en 2 dias. Responde con el numero del proveedor o 'cancelar'."

• Auto-cancelacion: Si no hay respuesta en 72 horas (vigencia de cotizacion), cancelar automaticamente el pedido: "Tu solicitud de cotizacion expiro. Cuando necesites material, mandame un nuevo pedido."

• Verificar vigencia: Antes de crear la orden, verificar que la cotizacion no haya expirado.

3.2 Respuesta ambigua del usuario

Escenario: El usuario dice "el mas barato pero que sea confiable" o "el que entregue mas rapido" o "no se, tu cual me recomiendas?"

Estado actual: interpretar_seleccion busca keywords como "mas barato" (retorna 1, el primero/mas barato), "primero", "segundo", etc. Si no matchea, remuestra las opciones.

Problemas detectados:

- "El que entregue mas rapido" no tiene handler. Necesitaria comparar tiempos de entrega.
- "Tu cual me recomiendas?" no genera recomendacion, solo remuestra opciones.
- "El mas barato pero confiable" es una combinacion que no se maneja.

Estrategias propuestas:

- Recomendacion inteligente: Si el usuario pide recomendacion, usar un scoring: precio (40%) + calificacion (30%) + tiempo de entrega (20%) + pedidos cumplidos (10%). Responder: "Te recomiendo Proveedor X: buen precio, calificacion 4.8/5, y entrega en 3 horas."
- Mas keywords: Agregar "mas rapido" (ordenar por tiempo_entrega), "mejor calificado" (ordenar por calificacion), "el confiable" (ordenar por pedidos_cumplidos).
- Pasar a Claude: Si la seleccion es compleja, usar Claude para interpretar la preferencia y mapearla a un proveedor.

3.3 Usuario quiere negociar

Escenario: "Puedes pedirle al proveedor 1 que baje su precio?" o "Dile que si me lo deja a 180 el bulto, va."

Estado actual: No existe funcionalidad de negociacion. El usuario solo puede seleccionar o cancelar.

Problemas detectados:

- La negociacion es parte fundamental de la compra de materiales en Mexico. No soportarla es una limitacion grande.
- El usuario tendria que contactar al proveedor directamente, perdiendo la trazabilidad.

Estrategias propuestas:

- Negociacion basica v1: Detectar keywords de negociacion ("baje", "descuento", "si me lo deja a", "contraoferta"). Enviar al proveedor: "El cliente te pide si puedes ajustar el precio a $X. Es posible?" Registrar como nueva ronda de cotizacion.
- Limite de rondas: Maximo 2 rondas de negociacion por proveedor para no abusar.
- Negociacion automatica: Si el usuario dice "preguntale a todos si pueden bajar", enviar a todos: "El cliente busca un mejor precio. Cual es tu ultima oferta?"

3.4 Precio cambio entre cotizacion y seleccion

Escenario: El proveedor cotizo $195/bulto ayer, el usuario acepta hoy, pero el proveedor dice "ya subio a $210".

Estado actual: La orden se crea con el precio de la cotizacion. No hay confirmacion del proveedor al momento de aceptacion.

Problemas detectados:

- Hay un gap entre la cotizacion y la confirmacion donde el precio puede cambiar.
- El proveedor no recibe notificacion cuando el cliente lo elige. No sabe que tiene un pedido.
- Si el proveedor se niega a respetar el precio, no hay mecanismo de resolucion.

Estrategias propuestas:

• Notificar al proveedor al ser elegido: INMEDIATAMENTE despues de que el usuario elige, enviar al proveedor: "Tu cotizacion fue aceptada! Orden #{id}, {items}, Total: ${total}. Confirmas disponibilidad y precio?" La orden deberia estar en status "pendiente_confirmacion_proveedor" hasta que el proveedor confirme.
• Vigencia estricta: Si la cotizacion ya paso su vigencia de 3 dias, no permitir crear la orden. Pedir re-cotizacion.
• Penalizacion por incumplimiento: Si un proveedor cotiza y luego dice que el precio cambio, bajar su calificacion automaticamente.

3.5 Cambio de opinion inmediato

Escenario: El usuario elige "1" y 30 segundos despues dice "no, mejor el 2".

Estado actual: La orden ya se creo con proveedor 1. El segundo mensaje se procesaria como contexto "orden_activa". No hay forma de cambiar la seleccion.

Problemas detectados:

• La orden se crea instantaneamente sin periodo de gracia.
• Cancelar y crear otra orden dejaria registros huerfanos.
• El proveedor 1 ya fue notificado (potencialmente).

Estrategias propuestas:

• Periodo de gracia de 5 minutos: Despues de crear la orden, esperar 5 minutos antes de notificar al proveedor. Si el usuario cambia de opinion en ese periodo, cambiar la orden sin costo.
• Comando de cambio: Detectar "no, mejor el 2" o "cambie de opinion, quiero el otro". Cancelar orden actual y crear nueva.
• Confirmacion doble para montos grandes: Si la orden es >$50,000 MXN, pedir confirmacion: "Vas a confirmar un pedido por $85,000 con Proveedor X. Seguro? Responde SI para confirmar."

PROCESO 4: APROBACION CORPORATIVA

4.1 Aprobador de vacaciones

Escenario: El unico aprobador con limite suficiente esta de vacaciones y no revisa WhatsApp.

Estado actual: La aprobacion tiene expiracion de 24 horas (EXPIRACION_HORAS = 24). Se envian hasta 3 recordatorios (MAX_RECORDATORIOS = 3). Pero verificar_expiradas necesita ejecutarse desde un scheduler que no esta claramente configurado.

Problemas detectados:

- CRITICO: verificar_expiradas no se llama desde ningun lugar automaticamente. Las aprobaciones podrian nunca expirar en la practica.
- No hay escalacion a un aprobador alternativo.
- Si expira, no se notifica al solicitante de que su pedido fue rechazado/expirado.
- No hay concepto de "aprobador suplente" o "delegacion de autoridad".

Estrategias propuestas:

- URGENTE - Agregar al agente proactivo: Llamar verificar_expiradas en cada ciclo del agente proactivo. Cuando una aprobacion expire, notificar al solicitante: "Tu solicitud de aprobacion para orden #{id} expiro porque no recibio respuesta en 24h. Opciones: 1) Re-enviar la solicitud, 2) Cancelar."
- Aprobador suplente: Agregar campo suplente_id a MiembroEmpresa. Si el aprobador principal no responde en 12h, escalar al suplente.
- Aprobacion por email: Si no responde por WhatsApp en 6h, enviar email con link de aprobacion (one-click approve/reject).
- Auto-aprobacion por monto bajo: Permitir configurar que montos menores a X (ej: $5,000) se auto-aprueben sin intervencion.

4.2 Sin aprobadores elegibles

Escenario: La empresa requiere aprobacion pero ningun miembro tiene puede_aprobar = True o el monto excede el limite de todos los aprobadores.

Estado actual: solicitar_aprobacion busca aprobadores con limite suficiente. Si ninguno lo tiene, el loop de envio simplemente no envia nada. La aprobacion se crea pero nadie la recibe.

Problemas detectados:

- La aprobacion queda colgada sin notificar a nadie.
- El solicitante recibe "requiere aprobacion, ya notifique a los responsables" pero nadie fue notificado.
- No hay validacion previa de que existan aprobadores.

Estrategias propuestas:

- Validacion previa: Antes de crear la aprobacion, verificar que exista al menos 1 aprobador con limite suficiente. Si no, informar al solicitante: "Tu empresa requiere aprobacion para compras de $X, pero no hay aprobadores configurados con ese limite. Contacta a tu administrador."
- Admin fallback: Si no hay aprobadores, enviar al admin de la empresa (si existe) para que configure uno.
- Log de alerta: Registrar en logs y dashboard que una empresa tiene un gap de aprobacion.

4.3 Aprobador responde despues de expiracion

Escenario: El aprobador responde "APROBAR 42" pero la aprobacion ya expiro.

Estado actual: manejar_aprobacion busca aprobaciones con status == "pendiente". Si ya expiro, el status seria "expirada" y no se encontraria. Se responderia: "No encontre solicitud pendiente para la orden #42."

Problemas detectados:

- El aprobador no entiende por que no funciona su aprobacion.
- No se le informa que la solicitud expiro.
- El pedido podria seguir siendo necesario.

Estrategias propuestas:

- Mensaje informativo: Buscar tambien aprobaciones expiradas y si la encuentra, responder: "Esta solicitud expiro hace X horas porque no recibio respuesta a tiempo. El solicitante puede volver a pedirla."
- Reactivacion: Permitir al aprobador reactivar una aprobacion expirada: "La solicitud habia expirado, pero la reactive con tu aprobacion. El solicitante sera notificado."

4.4 Formato incorrecto de aprobacion

Escenario: El aprobador responde "si dale" o "aprobado" en lugar de "APROBAR 42".

Estado actual: es_aprobacion busca exactamente ^APROBAR\s+(\d+). "si dale" no matchea, asi que el mensaje se procesaria como un mensaje normal. Si el aprobador no tiene pedidos activos, iria al flujo "nuevo_pedido" y Claude intentaria interpretarlo como un pedido de materiales.

Problemas detectados:

- MUY COMUN: Los aprobadores (directores, gerentes) no van a escribir "APROBAR 42" exactamente. Van a escribir "aprobado", "si", "dale", "ok", "esta bien".
- El sistema no identifica al aprobador como tal si no usa el formato exacto.
- El aprobador podria recibir "Hola, soy Nico. Que material necesitas?" lo cual seria muy confuso.

Estrategias propuestas:

- Deteccion inteligente de aprobacion: Si un usuario tiene aprobaciones pendientes y responde con algo que suena afirmativo ("si", "dale", "ok", "aprobado"), preguntarle: "Tienes una aprobacion pendiente por $X para orden #{id}. Estas aprobando? Responde APROBAR {id} para confirmar."
- Formato simplificado: Si solo tiene 1 aprobacion pendiente, aceptar "si", "dale", "aprobado" como aprobacion directa sin necesitar el numero.
- Botones de WhatsApp: Usar interactive messages con botones "Aprobar" y "Rechazar" para que no tengan que escribir nada.

4.5 Dos aprobadores responden al mismo tiempo

Escenario: Dos directores reciben la solicitud y ambos escriben "APROBAR 42" simultaneamente.

Estado actual: aprobar_orden busca aprobacion con status == "pendiente". El primero en llegar a la BD la cambia a "aprobada". El segundo no la encontraria (ya no es "pendiente").

Problemas detectados:

- Race condition potencial: si ambas queries llegan en el mismo milisegundo, podrian ambas encontrar la aprobacion como "pendiente" y ambas aprobarla.
- El segundo aprobador recibiria "No se pudo aprobar la orden #42" sin explicacion clara.

Estrategias propuestas:

- Lock de BD: Usar SELECT FOR UPDATE al buscar la aprobacion para evitar race conditions.
- Mensaje informativo: Si la aprobacion ya fue procesada, decir: "La orden #42 ya fue aprobada por {nombre_aprobador_anterior} hace X minutos."
- Registro de ambas respuestas: Aunque solo una se procese, registrar que ambos aprobadores respondieron (para auditoria).

4.6 Telefono del solicitante cambio

Escenario: El residente que pidio el material cambio de numero y la notificacion de aprobacion/rechazo no le llega.

Estado actual: Se busca al solicitante por aprobacion.solicitante_id y se usa su telefono actual. Si el telefono cambio en la BD, deberia funcionar. Pero si cambio de numero sin actualizar, el mensaje va al numero viejo.

Problemas detectados:

- No hay forma de que el usuario actualice su numero en el sistema.
- WhatsApp no rebota mensajes a numeros que cambiaron (simplemente no se entregan).

Estrategias propuestas:

- Verificacion periodica: Cada mes, enviar un "ping" a los usuarios activos para verificar que su numero funciona.
- Notificacion por multiples canales: Si la aprobacion es critica, intentar tambien por email (si esta registrado).
- Auto-deteccion: Si un mensaje no se entrega (status "sent" pero nunca "delivered"), marcar el usuario para revision.

4.7 Empresa sin presupuesto configurado

Escenario: La empresa requiere aprobacion pero no tiene presupuesto (PresupuestoObra) configurado.

Estado actual: En componer_mensaje_aprobacion, se busca el presupuesto activo. Si no existe, simplemente no se muestra la seccion de alerta presupuestal. La aprobacion funciona normalmente.

Problemas detectados:

- El aprobador no tiene contexto de cuanto ha gastado la empresa, asi que aprueba a ciegas.

- Deberia haber al menos un aviso de que no hay presupuesto configurado.

Estrategias propuestas:

- Advertencia: Si no hay presupuesto, incluir en el mensaje de aprobacion: "Esta empresa no tiene presupuesto de obra configurado. Se recomienda establecer uno para mejor control."
- Configuracion guiada: Ofrecer al admin configurar un presupuesto la primera vez.

PROCESO 5: ORDEN Y ENTREGA

5.1 EL GRAN PROBLEMA: Proveedor acepta, se queda en silencio, cliente pregunta

Escenario: Se crea la orden, el proveedor fue notificado (o no), y no hay actualizaciones. El cliente manda "donde va mi pedido?" y el status sigue en "confirmada" 8 horas despues.

Estado actual:

- recordar_proveedor_compromiso alerta al proveedor a las 4h y 12h si no ha actualizado a "preparando".
- alertar_retraso_entrega alerta a ambos (usuario y proveedor) a 1h, 4h y 12h despues de la fecha prometida.
- Cuando el usuario pregunta status, recibe el status actual (que sigue siendo "confirmada").

Problemas detectados:

- CRITICO: No se notifica al proveedor que fue elegido. La orden se crea internamente pero el proveedor NO recibe un mensaje de "tienes un pedido confirmado". El proveedor puede no saber que tiene una orden pendiente.
- No hay mecanismo de escalacion automatica (buscar proveedor alternativo).
- Despues de 12h de retraso, el sistema sigue solo mandando mensajes. No toma accion.
- No hay "boton de panico" para que el usuario escale manualmente.
- Si la fecha de entrega prometida no se registro (campo nulo), las alertas de retraso no se activan.

Estrategias propuestas:

- URGENTE - Notificar al proveedor: Al crear la orden, INMEDIATAMENTE enviar WhatsApp al proveedor: "Un cliente acepto tu cotizacion! Orden #{id}: {items}. Total: ${total}. Entrega en: {direccion}. Confirma que puedes cumplir respondiendo PREPARANDO {id}."
- Escalacion automatica por niveles:
  - Nivel 1 (2h sin update): Recordatorio al proveedor: "Tienes una orden pendiente, actualiza el status."
  - Nivel 2 (6h sin update): Alerta al usuario: "Tu proveedor no ha actualizado el status. Estamos en contacto con el."
  - Nivel 3 (12h sin update): Alerta al admin de ObraYa + mensaje al usuario: "Detectamos un posible problema con tu pedido. Quieres que busquemos un proveedor alternativo?"
  - Nivel 4 (24h sin update): Auto-cancelar orden y re-cotizar automaticamente con proveedores alternativos.

- Comando de escalacion: El usuario puede escribir "URGENTE" o "escalar" para activar busqueda de proveedor alternativo inmediatamente.
- Dashboard de ordenes criticas: Admin ve en tiempo real todas las ordenes con retraso.

5.2 Chofer perdido / no encuentra la direccion

Escenario: El chofer llega a la zona pero no encuentra la obra. Llama al numero del residente pero no contesta.

Estado actual: Si el proveedor actualiza status a "en_transito", el usuario recibe datos del chofer (nombre, telefono, placas). Pero no hay mecanismo de comunicacion directa chofer-residente a traves de la plataforma.

Problemas detectados:

- El telefono del chofer se muestra al usuario pero no viceversa (el chofer no tiene el numero del residente).
- No hay GPS tracking del chofer.
- Si el chofer no encuentra la obra, no hay proceso definido.

Estrategias propuestas:

- Compartir datos bidireccionales: Al enviar datos del chofer al usuario, tambien enviar al chofer: "Nombre del contacto en obra: {nombre}. Telefono: {telefono}. Direccion: {direccion}. Pin de ubicacion: {link}"
- Enviar ubicacion GPS: Si se tiene la ubicacion de la obra, enviarla como pin de WhatsApp al chofer.
- Canal de soporte: Si el chofer no encuentra, que pueda mandar WhatsApp a Nico: "No encuentro la obra de la orden #{id}". Nico contacta al residente.

5.3 Entrega parcial

Escenario: Se pidieron 15 m3 de concreto pero el camion trajo solo 10 m3. El proveedor dice "el resto te lo llevo manana".

Estado actual: La logica de confirmacion es binaria: OK (confirmar entrega) o Problema (crear incidencia). No hay concepto de entrega parcial.

Problemas detectados:

- Si el usuario confirma, la orden se marca como "entregada" cuando en realidad falta material.
- Si reporta problema, se crea incidencia pero no se trackea la segunda entrega.
- No hay forma de registrar multiples entregas para una misma orden.

Estrategias propuestas:

- Entregas parciales: Agregar status "entrega_parcial". El usuario confirma lo recibido y se registra cuanto falta. La orden queda abierta hasta completar.
- Sub-ordenes de entrega: Cada viaje del camion es una sub-entrega con su propio registro: fecha, cantidad, chofer, confirmacion.
- Seguimiento de complemento: Si hay entrega parcial, el agente proactivo debe dar seguimiento: "Falta 1/3 de tu

pedido. El proveedor quedo de entregarlo manana. Te recuerdo a las 8am."

5.4 Material equivocado

Escenario: Pidieron cemento CPC 40R y llego CPC 30R.

Estado actual: El usuario reporta problema. clasificar_incidencia detectaria keywords "equivocado", "no era", "diferente" y clasificaria como tipo = "especificacion" con severidad = "grave".

Problemas detectados:

- Se registra la incidencia pero no hay flujo para resolver: devolucion? reenvio del material correcto? credito?
- El usuario queda con material que no sirve para su obra.
- No se contacta automaticamente al proveedor sobre la incidencia.

Estrategias propuestas:

- Notificar al proveedor: Cuando se crea una incidencia de especificacion, enviar inmediatamente al proveedor: "El cliente reporta que recibio material incorrecto. Orden #{id}: pidio {esperado}, recibio {recibido}. Contacta al cliente para resolver."
- Opciones de resolucion al usuario: "Reportamos el problema al proveedor. Opciones: 1) Devolucion y reenvio del material correcto, 2) Descuento sobre el material recibido, 3) Cancelacion y re-cotizacion con otro proveedor."
- Tracking de resolucion: La incidencia debe tener un flujo de resolucion con timestamps.

5.5 Material danado

Escenario: Las varillas llegaron torcidas o los bultos de cemento llegaron mojados/abiertos.

Estado actual: Similar al caso anterior: incidencia clasificada como "material_danado", severidad "media".

Problemas detectados:

- Severidad "media" para material danado es muy baja. Material danado puede significar miles de pesos de perdida.
- No hay evidencia fotografica. El usuario dice "llego danado" pero no hay prueba.

Estrategias propuestas:

- Subir severidad: Material danado deberia ser "grave" por defecto, especialmente para montos > $10,000.
- Evidencia fotografica: Pedir al usuario: "Mandame foto del material danado para documentar la incidencia."
- Seguro de envio: Para pedidos grandes, ofrecer seguro de transporte. Si se dana, el seguro cubre.

5.6 Nadie en la obra para recibir

Escenario: El camion llega a la obra a las 2pm pero el residente que pidio ya se fue. No hay nadie que firme o verifique la entrega.

Estado actual: No hay manejo de este caso. El proveedor probablemente marcaria "en_obra" o se iria con el material.

Problemas detectados:

- Material dejado sin supervision puede ser robado.
- El proveedor pierde tiempo y combustible.
- No hay concepto de "contacto alterno en obra".

Estrategias propuestas:

- Contacto alterno: Al crear el pedido, pedir un segundo contacto en obra (maestro, velador, otro residente).
- Confirmacion de horario: Antes de la entrega, confirmar: "Tu material sale en 30 minutos. Habra alguien para recibir?"
- Protocolo de no-contacto: Si nadie recibe, el chofer manda foto de la obra cerrada a Nico. Nico contacta al usuario. Si no contesta en 30 min, reprogramar entrega.

5.7 Lluvia / condiciones climaticas

Escenario: El concreto premezclado esta en camino pero empezo a llover y no se puede colar.

Estado actual: No hay manejo de condiciones climaticas.

Problemas detectados:

- Concreto premezclado tiene vida util de ~90 minutos. Si no se puede colar, se pierde.
- No hay opcion de cancelar/posponer una orden en transito.
- El costo del concreto perdido es del usuario, no del proveedor.

Estrategias propuestas:

- Alerta climatica: Integrar API de clima. Si hay probabilidad >70% de lluvia en la zona de entrega, alertar al usuario: "Hay pronostico de lluvia para tu zona. Quieres reprogramar la entrega del concreto?"
- Cancelacion en transito: Permitir cancelar con penalizacion parcial (flete + % de material para perecederos como concreto).
- Reprogramacion rapida: Boton de "Reprogramar para manana" que notifica al proveedor.

5.8 Proveedor dice "entregado" pero cliente dice "nunca llego"

Escenario: Disputa entre proveedor y cliente sobre si la entrega se hizo o no.

Estado actual: Si el proveedor marca "en_obra" pero el usuario no confirma, el agente proactivo insiste a las 2h y 6h. Pero no hay resolucion formal del conflicto.

Problemas detectados:

- Escenario de fraude potencial (por cualquiera de las dos partes).
- No hay evidencia de entrega (foto, firma, GPS del chofer en la ubicacion).

• No hay proceso de arbitraje.

Estrategias propuestas:

• Evidencia de entrega: Requerir al chofer: foto de la descarga, nombre de quien recibe, y ubicacion GPS al momento de la entrega.
• Firma digital: El residente confirma entrega con un codigo unico que solo tiene el, que le muestra al chofer.
• Arbitraje: Si hay disputa, ObraYa interviene como mediador con la evidencia disponible. Escalar a admin.
• Retencion de pago: No liberar el pago al proveedor hasta que el usuario confirme. Si hay disputa, retener hasta resolucion.

5.9 Rechazo de entrega

Escenario: El material llega pero el residente dice "no acepto, esta mal" y rechaza la entrega.

Estado actual: No hay flujo de rechazo de entrega. Solo hay confirmacion o incidencia.

Problemas detectados:

• Si el usuario rechaza, el material se regresa al proveedor. Quien paga el flete de regreso?
• La orden queda en limbo.
• No se notifica automaticamente al proveedor que la entrega fue rechazada.

Estrategias propuestas:

• Status "rechazada_entrega": Nuevo status con flujo: el material se regresa, se crea incidencia automatica, se ofrece re-cotizacion.
• Foto obligatoria: Si rechaza, pedir foto del motivo de rechazo.
• Politica de costos: Si el rechazo es justificado (material incorrecto), el proveedor absorbe el flete. Si es injustificado, el usuario paga penalizacion.

5.10 Direccion incorrecta

Escenario: La direccion de entrega tiene error y el camion llega al lugar equivocado.

Estado actual: La direccion se captura del pedido original. No hay validacion de direccion.

Problemas detectados:

• Geocoding solo convierte GPS a texto, no valida que la direccion exista.
• No hay verificacion de la direccion con el usuario antes de enviar.
• Un error tipico: "Av Americas 1500" vs "Av Americas 15000" — numeros equivocados.

Estrategias propuestas:

• Confirmacion de direccion: Al crear la orden, enviar la direccion al usuario: "Confirma la direccion de entrega:

{direccion}. Es correcta? Si no, mandame la correcta o un pin de ubicacion."
• Google Maps link: Incluir un link a Google Maps de la direccion para verificacion visual.
• Correccion pre-envio: Permitir corregir la direccion hasta 30 minutos antes de que salga el camion.

5.11 Entrega en fin de semana / dia festivo

Escenario: El pedido se hace viernes a las 5pm, el proveedor no trabaja sabado/domingo, pero el usuario espera entrega para "manana" (sabado).

Estado actual: No hay validacion de dias habiles. Los tiempos de entrega del proveedor pueden ser "manana" pero no especifican si es dia habil.

Problemas detectados:

• El usuario espera sabado, el proveedor no trabaja hasta lunes. Incidencia de "retraso" injusta.
• No se registran los horarios de operacion de cada proveedor.

Estrategias propuestas:

• Horarios de proveedor: Agregar campos dias_operacion y horario_atencion a la tabla de proveedores.
• Validacion de fecha: Si el usuario pide entrega en dia no habil, avisar: "El proveedor no opera los sabados. La entrega mas temprana seria el lunes. Te parece?"
• Filtro por disponibilidad: En la cotizacion activa, solo contactar proveedores que operen el dia solicitado.

PROCESO 6: PAGOS

6.1 Stripe caido

Escenario: El usuario quiere pagar pero Stripe no responde.

Estado actual: crear_sesion_pago llamaria a stripe.checkout.Session.create que lanzaria una excepcion. No hay manejo de fallback.

Problemas detectados:

• No hay catch de excepciones de Stripe en el servicio de pagos.
• No hay metodo de pago alternativo si Stripe falla.
• El usuario queda sin poder pagar.

Estrategias propuestas:

• Try/catch con mensaje: Capturar excepciones de Stripe y responder: "El sistema de pagos esta temporalmente no

disponible. Intenta en unos minutos o paga por transferencia: Cuenta {datos}."

• Pago por transferencia: Ofrecer como alternativa permanente: CLABE interbancaria de ObraYa, con referencia = numero de orden.

• Conciliacion manual: Si paga por transferencia, validar manualmente y confirmar la orden.

6.2 Pago exitoso pero webhook de Stripe falla

Escenario: Stripe cobra al usuario pero la confirmacion via webhook no llega a nuestra BD. El usuario pago pero la orden no se marca como pagada.

Estado actual: confirmar_pago se llama desde algun endpoint de webhook de Stripe (no visible en los archivos leidos). Si falla, la orden queda sin marcar.

Problemas detectados:

• El usuario pago y el dinero salio de su cuenta, pero ObraYa no lo sabe.

• El proveedor no recibe pago porque ObraYa no proceso la confirmacion.

• No hay reconciliacion automatica.

Estrategias propuestas:

• Job de reconciliacion: Cada hora, revisar ordenes pendientes de pago. Para cada una, consultar Stripe API directamente: stripe.checkout.Session.list(metadata={"orden_id": X}). Si Stripe dice que ya se pago, confirmar en nuestra BD.

• Idempotencia: confirmar_pago ya verifica if orden.pagado: return orden, lo cual es bueno para evitar doble procesamiento.

• Alerta a admin: Si una orden tiene mas de 2 horas desde que se creo el checkout y no se confirmo pago, alertar.

6.3 Chargeback / disputa de pago

Escenario: El usuario hace un contracargo con su banco despues de pagar.

Estado actual: No hay manejo de chargebacks. El dinero se devuelve al usuario pero ObraYa ya pago al proveedor.

Problemas detectados:

• ObraYa pierde dinero: ya pago al proveedor (o se comprometio a pagar).

• No hay evidencia de entrega para disputar el chargeback.

• El material ya se uso en la obra.

Estrategias propuestas:

• Webhook de disputas: Implementar webhook de Stripe para charge.dispute.created. Al recibir, pausar orden, alertar admin.

• Evidencia de entrega: Usar la confirmacion del usuario (timestamp, mensaje "OK") como evidencia para disputar el

chargeback con Stripe.
• Blacklist: Si un usuario hace chargeback, bloquear para futuras compras. Su credit score baja a 0.
• Retencion de pago al proveedor: No pagar al proveedor hasta X dias despues de la entrega (periodo de disputa).

6.4 Pago en parcialidades / credito

Escenario: Un pedido de $200,000 que el usuario quiere pagar en 3 mensualidades.

Estado actual: Solo hay pago unico via Stripe Checkout. No hay soporte para pagos parciales.

Problemas detectados:

• Pedidos grandes son comunes en construccion ($100K-$500K).
• Muchas constructoras necesitan credito a 30/60/90 dias.
• El sistema de credit scoring existe pero no se usa para otorgar credito real.

Estrategias propuestas:

• v1 - Pagos parciales: Dividir el checkout en N pagos. Enviar links de pago con fechas.
• v2 - Linea de credito: Usar el credit scoring para otorgar linea de credito. Compra ahora, paga a 30 dias.
• v3 - Facturaje: Integrar con factoring companies (como Konfio, Credijusto) para ofrecer financiamiento al usuario.

6.5 Proveedor necesita pago antes de entregar (COD)

Escenario: El proveedor dice "pago contra entrega" o "necesito anticipo del 50%".

Estado actual: No hay soporte para pagos parciales ni pago contra entrega. El flujo asume que ObraYa cobra al usuario y luego paga al proveedor.

Problemas detectados:

• Muchos proveedores en Mexico trabajan con COD o anticipo. No soportarlo limita la base de proveedores.
• Si ObraYa paga al proveedor antes de cobrar al usuario, hay riesgo financiero.

Estrategias propuestas:

• Pago anticipado del usuario: Si el proveedor requiere COD, cobrar al usuario antes de confirmar la orden con el proveedor.
• Fondo de garantia ObraYa: ObraYa adelanta el pago al proveedor para usuarios con buen credit score (>80).
• Registrar condiciones de pago: Cada proveedor tiene campo condiciones_pago: "credito 30 dias", "COD", "anticipo 50%". Mostrar en la comparativa.

6.6 Facturacion (factura fiscal)

Escenario: El usuario necesita factura para deducir impuestos.

Estado actual: No hay soporte de facturacion. No se menciona CFDI, RFC, ni regimen fiscal.

Problemas detectados:

- En Mexico, las empresas constructoras NECESITAN factura para deducir materiales.
- Sin factura, los clientes corporativos no pueden usar ObraYa.
- El proveedor es quien factura al comprador, pero ObraYa cobra la comision.

Estrategias propuestas:

- Recopilar datos fiscales: Al registrar empresa, pedir RFC, razon social, regimen fiscal, uso CFDI.
- Solicitar factura al proveedor: Al crear la orden, indicar al proveedor: "El cliente requiere factura a nombre de: {razon_social}, RFC: {rfc}."
- Facturacion de la comision: ObraYa debe facturar su 2% de comision al usuario. Integrar con servicio de timbrado CFDI (Facturama, SW Sapien).

6.7 Discrepancia de precio

Escenario: La cotizacion fue $50,000 pero Stripe cobra $51,000 (por la comision del 2%).

Estado actual: calcular_desglose suma 2% de comision. El total que ve el usuario en Stripe es total + 2%.

Problemas detectados:

- El usuario podria no entender por que se le cobra mas de lo cotizado.
- La comision no se explica claramente antes del checkout.

Estrategias propuestas:

- Transparencia total: Antes de enviar al checkout, mostrar desglose: "Materiales: $50,000 / Servicio ObraYa (2%): $1,000 / Total a pagar: $51,000."
- Incluir comision en cotizacion: Considerar mostrar los precios ya con comision desde la comparativa para evitar sorpresas.

PROCESO 7: PRESUPUESTOS

7.1 Consumo excede presupuesto pero el material es urgente

Escenario: El presupuesto de cemento esta al 95% pero hay una emergencia en obra que requiere 50 bultos mas.

Estado actual: verificar_disponibilidad retorna permitido: False si la cantidad excede lo disponible. bloquear_partida se activa al 100%.

Problemas detectados:

- No hay mecanismo de override de emergencia.
- El residente necesita el material YA y no puede esperar a que el admin desbloquee.
- El bloqueo es absoluto: 100% = no puedes comprar mas, punto.

Estrategias propuestas:

- Override con aprobacion especial: Permitir la compra pero requerir aprobacion del director/admin. "La partida de cemento esta al 95%. Esta compra la excederia. Necesitas aprobacion especial de tu director."
- Margen de tolerancia: Permitir un 10% de sobregasto por partida antes de bloquear completamente.
- Alerta preventiva: A las compras 5 y 6 despues de la alerta del 80%, ser mas enfatico: "Si sigues comprando cemento, el presupuesto se agota en 2 pedidos mas."

7.2 Dos residentes piden lo mismo y juntos exceden presupuesto

Escenario: Residente A pide 500 bultos de cemento ($97,500) y Residente B pide 300 bultos ($58,500) al mismo tiempo. El presupuesto de cemento solo tiene $120,000 disponibles.

Estado actual: verificar_disponibilidad verifica por usuario individual. No hay lock global del presupuesto. Ambos pedidos pasarian la verificacion individual pero juntos exceden.

Problemas detectados:

- Race condition en el presupuesto: ambos pasan la verificacion pero el total excede.
- El presupuesto se asocia por usuario_id, no por empresa. Dos usuarios de la misma empresa podrian tener presupuestos separados.
- No hay reserva de presupuesto al momento de la cotizacion (solo se consume al crear la orden).

Estrategias propuestas:

- Reserva de presupuesto: Al iniciar cotizacion, reservar el monto estimado en el presupuesto. Si no se concreta, liberar la reserva.
- Presupuesto por empresa, no por usuario: El presupuesto deberia ser de la obra/empresa, no del usuario individual.
- Lock optimista: Al crear la orden, verificar de nuevo la disponibilidad con lock de BD para evitar race conditions.

7.3 Precios cambiaron desde la creacion del presupuesto

Escenario: El presupuesto se creo con cemento a $180/bulto pero ahora cuesta $210/bulto.

Estado actual: El presupuesto usa precio_unitario_estimado que se fija al momento de creacion. Los pedidos reales usan precios de mercado actuales.

Problemas detectados:

- El presupuesto se agota mas rapido de lo esperado porque los precios subieron.

- El porcentaje de consumo no refleja la realidad: 80% del presupuesto podria comprar solo 60% de los materiales necesarios.
- No hay re-estimacion automatica del presupuesto con precios actuales.

Estrategias propuestas:

- Re-estimacion periodica: Cada semana, recalcular el presupuesto con precios actuales de la BD. Si la diferencia es >10%, alertar: "Tu presupuesto de cemento subio un 15% por incremento de precios. Te sugerimos actualizar el presupuesto."
- Presupuesto por monto, no por cantidad: En lugar de "500 bultos a $180 = $90,000", presupuestar "$90,000 para cemento" y trackear cuanto se ha gastado en pesos.
- Comparacion plan vs real: Dashboard que muestre presupuesto original vs gasto real por partida.

PROCESO 8: CREDIT SCORING

8.1 Empresa nueva sin historial pero reputada

Escenario: Una constructora grande (ICA, GAYA) se registra en ObraYa por primera vez. Score = 50 (neutral, sin historial), no elegible para credito.

Estado actual: calcular_score da 50 a usuarios sin historial de pagos. Se necesitan 5 pedidos y score >= 65 para credito.

Problemas detectados:

- Discrimina injustamente a empresas grandes que son solventes.
- Una empresa que mueve $10M al mes tiene que hacer 5 pedidos pequenos antes de acceder a credito.
- No hay onboarding diferenciado por tamano de empresa.

Estrategias propuestas:

- Verificacion manual: Para empresas con RFC y razon social, verificar en Buro de Credito o DUNS. Si tiene buen historial externo, dar score inicial de 70.
- Fast-track: Permitir a empresas verificadas acceder a credito desde el primer pedido con garantia (carta compromiso, pagare).
- Score por empresa, no solo por usuario: El credit scoring deberia considerar el historial de TODOS los usuarios de la misma empresa.

8.2 Score cae dramaticamente por un evento aislado

Escenario: Un usuario con 50 pedidos perfectos tiene UN pago tarde (8 dias) y su score baja significativamente.

Estado actual: El ratio pagos a tiempo / totales pondera 40% del score. Un pago tarde de 50 cambia el ratio de 100% a 98%, bajando score_pagos de 100 a 98. Impacto en score total: -0.8 puntos. Esto es razonable.

Problemas detectados:

- El impacto real es menor de lo temido, pero para usuarios con pocos pedidos (5-10), un pago tarde si tiene impacto fuerte (de 100% a 80% ratio = -8 puntos en score total).
- No hay concepto de "perdon" o "amortiguacion" para el primer incidente.
- No se considera el contexto del pago tarde (dias festivos, problemas bancarios).

Estrategias propuestas:

- Gracia para primer incidente: El primer pago tarde no afecta el score si se paga dentro de 14 dias.
- Peso por recencia: Pagos recientes pesan mas que historicos. Un pago tarde hace 6 meses deberia importar menos que uno de la semana pasada.
- Explicacion al usuario: Si el score baja, notificar: "Tu score crediticio bajo de 85 a 78 porque tu ultimo pago tomo 12 dias. Pagos a tiempo mejoran tu score."

8.3 Fraude — alguien usando numero ajeno

Escenario: Alguien obtiene el WhatsApp de un residente y hace pedidos a su nombre, cargando a la cuenta de la empresa.

Estado actual: La unica autenticacion es el numero de WhatsApp. Quien tenga el numero puede hacer pedidos.

Problemas detectados:

- CRITICO de seguridad: No hay autenticacion real. Si alguien clona un numero o roba un telefono, puede hacer pedidos.
- No hay limites de gasto por sesion.
- No hay verificacion de identidad.

Estrategias propuestas:

- PIN de seguridad: Al registrarse, el usuario elige un PIN de 4 digitos. Para pedidos > $10,000, se pide el PIN.
- Alertas de actividad inusual: Si un usuario hace 5 pedidos en 1 hora, o un pedido 10x mayor al promedio, alertar al admin de la empresa.
- Verificacion two-factor: Para cambios de limites o datos, enviar codigo de verificacion por SMS a un segundo numero registrado.
- Limites de gasto diario: Configurar un limite maximo de compra por dia por usuario.

PROCESO 9: CALIFICACIONES

9.1 Usuario no responde a solicitud de calificacion

Escenario: Se pide calificacion 24h despues de la entrega y el usuario no responde.

Estado actual: pedir_calificacion_post_entrega envia solicitud en la ventana 24-25h despues de entrega. No hay reintentos.

Problemas detectados:

- Solo hay 1 oportunidad de pedir calificacion (ventana de 1 hora).
- La mayoria de los usuarios no calificaran sin incentivo.
- Sin calificaciones, el sistema de ranking de proveedores no funciona.

Estrategias propuestas:

- Segundo recordatorio: Pedir calificacion a las 48h si no respondio a las 24h.
- Calificacion automatica: Si no responde en 72h, asignar calificacion automatica basada en metricas medibles (puntualidad, incidencias).
- Incentivos: "Califica tu pedido y recibe 1% de descuento en tu proxima compra."
- Simplificar: En lugar de 1-5 estrellas, preguntar: "Tu pedido llego bien? Responde SI o NO."

9.2 Calificacion injusta (1 estrella)

Escenario: El usuario da 1 estrella porque el material "tardo mucho" pero en realidad llego en el tiempo prometido — el usuario esperaba algo irreal.

Estado actual: Las calificaciones manuales del usuario se guardan pero hay una calificacion automatica basada en metricas objetivas (puntualidad, concordancia, etc.).

Problemas detectados:

- La calificacion manual y la automatica podrian diferir mucho.
- No hay forma de que el proveedor conteste/dispute una calificacion injusta.
- Una calificacion de 1 estrella baja significativamente el promedio del proveedor.

Estrategias propuestas:

- Peso ponderado: Si la calificacion automatica (basada en datos) difiere mucho de la manual, usar un promedio ponderado: 60% automatica + 40% manual.
- Contestacion del proveedor: Permitir al proveedor responder a calificaciones negativas. No se publica al usuario pero el admin lo ve.
- Deteccion de outliers: Si la calificacion manual es 1 pero todos los indicadores objetivos son buenos, flaggear para revision manual.
- Minimo de calificaciones: No mostrar promedio hasta tener 5+ calificaciones. Un solo 1 estrella no deberia definir a un proveedor.

PROCESO 10: GESTION DE EQUIPOS

10.1 Empleado sale de la empresa

Escenario: Un residente renuncia pero su WhatsApp sigue registrado en ObraYa bajo la empresa.

Estado actual: No hay proceso de baja de empleados. El usuario seguiria pudiendo hacer pedidos a nombre de la empresa.

Problemas detectados:

- Ex-empleado podria hacer pedidos fraudulentos.
- Las ordenes activas del empleado quedan huerfanas.
- El presupuesto que tenia asignado no se libera.

Estrategias propuestas:

- Panel de admin de empresa: El admin puede dar de baja miembros. Al dar de baja: desactivar en MiembroEmpresa, cancelar ordenes pendientes, notificar al admin.
- Expiracion de membresia: Si un miembro no hace actividad en 90 dias, enviarlo a revision.
- Transferencia de ordenes: Las ordenes activas del empleado que sale se transfieren a otro miembro designado.

10.2 Promocion de empleado

Escenario: Un residente es promovido a superintendente y necesita limites de compra mayores.

Estado actual: MiembroEmpresa tiene limite_aprobacion y puede_aprobar. Cambiar estos campos requiere acceso directo a la BD.

Problemas detectados:

- No hay interfaz para gestionar roles y limites.
- Cambios de roles no se reflejan en el credit scoring individual.

Estrategias propuestas:

- Gestion via WhatsApp para admins: El admin de la empresa puede escribir: "SUBIR LIMITE {telefono} {nuevo_limite}" para cambiar el limite de compra.
- Roles predefinidos: Residente ($50K), Superintendente ($150K), Director ($500K). Asignar roles en lugar de limites manuales.
- Audit trail: Registrar todos los cambios de permisos con fecha, quien lo hizo, y motivo.

10.3 Proveedor cierra operaciones

Escenario: Un proveedor quiebra o cierra temporalmente. Tiene ordenes activas.

Estado actual: Si se marca activo = False, no aparece en futuras cotizaciones. Pero las ordenes activas quedan sin proveedor funcional.

Problemas detectados:

- Las ordenes activas quedan colgadas sin resolucion.
- Los precios historicos del proveedor dejan de ser referencia util.
- Los usuarios con ordenes pendientes no son notificados.

Estrategias propuestas:

- Proceso de cierre: Al desactivar un proveedor, automaticamente: (1) identificar ordenes activas, (2) notificar a usuarios afectados, (3) ofrecer re-cotizacion con otros proveedores, (4) cancelar solicitudes pendientes.
- Status "fuera_de_servicio": Diferente de "inactivo". Temporal vs permanente.
- Sustitucion automatica: Para ordenes en status "confirmada" (aun no preparada), re-cotizar automaticamente y presentar alternativas al usuario.

PROCESO 11: INTELIGENCIA DE PRECIOS

11.1 Manipulacion de precios

Escenario: Un proveedor sabe que ObraYa trackea precios y artificialmente infla sus cotizaciones para subir el "precio de mercado" registrado.

Estado actual: registrar_precios_desde_respuesta guarda cada precio cotizado en el historico. Si un proveedor infla precios, estos datos contaminan los promedios.

Problemas detectados:

- Un solo proveedor puede distorsionar el "precio de mercado" si tiene muchas cotizaciones.
- No hay deteccion de outliers en precios historicos.
- Los precios de referencia podrian ser artificialmente altos.

Estrategias propuestas:

- Deteccion de outliers: Al registrar un precio, comparar con la mediana historica. Si esta >25% arriba de la mediana, flaggear como "precio_sospechoso" y no incluirlo en el calculo del promedio.
- Precios ponderados por proveedor: Limitar la influencia de un solo proveedor al 20% del promedio. Si tiene muchas cotizaciones, las mas recientes se promedian.
- Validacion cruzada: Comparar con indices oficiales (CMIC, BIMSA) para detectar desviaciones del mercado.

11.2 Estacionalidad

Escenario: El precio del cemento sube en temporada de lluvias (menor produccion) y baja en temporada seca. Los promedios anuales no reflejan esto.

Estado actual: Los precios historicos se guardan con fecha pero no se analizan por temporada.

Problemas detectados:

- Los precios de referencia en enero podrian ser irrelevantes en agosto.
- Las alertas de "precio arriba del mercado" podrian ser falsos positivos en temporada alta.

Estrategias propuestas:

- Promedios por periodo: Calcular promedios moviles de 30 dias, no anuales.
- Tendencia: Mostrar al usuario "Este material ha subido 12% en el ultimo mes" para dar contexto.
- Proyeccion: Si hay tendencia alcista, recomendar: "El cemento esta subiendo. Considera comprar ahora antes de que suba mas."

PROCESO 12: COMUNICACION WHATSAPP

12.1 WhatsApp Cloud API caida

Escenario: Meta tiene una falla y la API de WhatsApp no responde por horas.

Estado actual: Hay fallback a Twilio (/webhook/twilio). Pero el fallback es para RECIBIR, no para ENVIAR. enviar_mensaje_texto usa la API de Meta directamente.

Problemas detectados:

- CRITICO: Si Meta esta caido, NO se pueden enviar mensajes. Todas las cotizaciones, notificaciones y alertas fallan silenciosamente.
- Los mensajes no se encolan para reenvio cuando se recupere.
- No hay health check de la API.

Estrategias propuestas:

- Fallback de envio a Twilio: Si enviar_mensaje_texto via Meta falla, reintentar via Twilio WhatsApp API.
- Cola de mensajes: Usar una cola (Redis, SQS) para mensajes pendientes. Si el envio falla, encolar y reintentar cada 5 minutos.
- Health check: Cada 5 minutos, enviar un mensaje de test a un numero interno. Si falla, activar modo degradado y alertar admin.

- Email como backup: Para notificaciones criticas (ordenes, pagos), enviar tambien por email si WhatsApp falla.
- SMS como ultimo recurso: Para alertas urgentes (retraso de entrega, problema de pago), usar SMS como backup si WhatsApp y email fallan.

12.2 Ventana de 24 horas cerrada

Escenario: Queremos contactar a un proveedor pero la ultima conversacion fue hace mas de 24 horas. WhatsApp Business no permite mensajes libres fuera de la ventana.

Estado actual: Se intenta con template primero, si falla se intenta texto libre. Si ambos fallan, la solicitud queda como "error_envio".

Problemas detectados:

- Si el template "solicitud_cotizacion" no esta aprobado por Meta, no se puede contactar a proveedores fuera de ventana.
- Los templates tardan 1-3 dias en ser aprobados. Si se rechazan, no hay plan B.
- No todos los mensajes pueden ser templates (las notificaciones de status, recordatorios, etc. son texto libre).

Estrategias propuestas:

- Aprobar multiples templates: Tener templates aprobados para cada caso: solicitud_cotizacion, recordatorio_cotizacion, orden_confirmada, notificacion_entrega, solicitud_calificacion.
- Mantener ventana abierta: Si un proveedor es activo, enviarle al menos 1 template por semana para mantener la ventana abierta.
- Notificacion por template: Convertir las notificaciones criticas a format de template para que funcionen fuera de ventana.

12.3 Rate limits de WhatsApp Business

Escenario: ObraYa crece y empieza a enviar miles de mensajes diarios, excediendo el tier de la cuenta.

Estado actual: No hay monitoreo de limites.

Problemas detectados:

- Tier 1: 250 conversaciones/24h. Tier 2: 1000. Tier 3: 10,000. Tier 4: ilimitado.
- Si se excede el tier, los mensajes se rechazan sin opcion de encolar.
- Los tiers se suben automaticamente con buen quality rating, pero una mala calificacion de calidad puede bajar el tier.

Estrategias propuestas:

- Monitoreo de uso: Dashboard que muestre: conversaciones abiertas hoy / limite del tier. Alerta al 70%.
- Priorizacion de mensajes: Si estamos cerca del limite, priorizar: ordenes activas > cotizaciones > recordatorios > marketing.
- Quality rating: Monitorear la calificacion de calidad de WhatsApp. Si baja, reducir mensajes proactivos para mejorarla.

12.4 Usuario bloquea a Nico

Escenario: El usuario bloquea el numero de ObraYa/Nico.

Estado actual: Los mensajes se envian pero nunca se entregan. El sistema no sabe que fue bloqueado.

Problemas detectados:

- Las ordenes activas del usuario no se pueden comunicar.
- El agente proactivo sigue intentando enviar mensajes inutiles.
- No hay canal alternativo.

Estrategias propuestas:

- Deteccion de bloqueo: Si multiples mensajes consecutivos no se entregan (status "sent" pero nunca "delivered"), marcar usuario como "comunicacion_bloqueada".
- Canal alternativo: Si detectamos bloqueo, intentar por email o SMS: "Detectamos que no podemos comunicarnos contigo por WhatsApp. Tienes ordenes activas. Contactanos al {telefono_soporte}."
- Pausa de ordenes: Si no se puede contactar al usuario, pausar ordenes pendientes hasta restablecer comunicacion.

PROCESO 13: AGENTE PROACTIVO

13.1 Scheduler deja de ejecutarse

Escenario: El servidor se reinicia, el cron job falla, o el proceso del agente proactivo se cuelga.

Estado actual: ejecutar_ciclo_agente esta disenado para correr cada 10-15 minutos. No se ve la configuracion del scheduler en los archivos leidos.

Problemas detectados:

- CRITICO: No esta claro que scheduler ejecuta el agente proactivo. Si es un cron de sistema o un scheduler de Python (APScheduler), ambos pueden fallar silenciosamente.
- Si el agente no corre, TODAS las funciones proactivas dejan de funcionar: recordatorios, alertas de retraso, calificaciones, timeout de cotizaciones.
- No hay alerta si el agente deja de correr.

Estrategias propuestas:

- Heartbeat monitoring: El agente debe registrar un timestamp en la BD cada vez que corre. Un servicio externo (o el propio health check) verifica que el ultimo heartbeat no tenga mas de 20 minutos.
- Alerta por caida: Si el heartbeat esta atrasado, enviar SMS/email al admin: "El agente proactivo de ObraYa no ha

corrido en 30 minutos. Revisar servidor."
• Scheduler redundante: Usar un servicio de scheduling externo (AWS EventBridge, Celery Beat) ademas del cron local.
• Auto-recovery: Si el proceso muere, systemd/supervisor lo reinicia automaticamente.

13.2 Demasiadas notificaciones — usuarios se hartan

Escenario: Un usuario con 3 ordenes activas recibe: recordatorio de confirmacion, alerta de retraso, solicitud de calificacion, y notificacion de presupuesto — todo en el mismo dia.

Estado actual: Cada funcion del agente proactivo opera independientemente. No hay agregacion ni limite global de mensajes por usuario.

Problemas detectados:

• Un usuario podria recibir 10+ mensajes al dia de ObraYa.
• Exceso de mensajes lleva a bloqueo de numero y mala experiencia.
• WhatsApp Business puede penalizar cuentas que generan muchos bloqueos.

Estrategias propuestas:

• Rate limit por usuario: Maximo 5 mensajes proactivos por usuario por dia. Si se excede, encolar para manana.
• Agregacion de mensajes: Si hay multiples notificaciones para el mismo usuario, combinarlas en un solo mensaje: "Actualizacion de tus pedidos: Orden #1 en transito, Orden #2 confirmada, Presupuesto al 80%."
• Horario de silencio: No enviar mensajes proactivos entre 9pm y 7am. Las notificaciones criticas (entrega en camino) si se envian siempre.
• Opt-out parcial: Permitir al usuario decir "no me mandes recordatorios" para desactivar notificaciones no criticas.

13.3 Notificaciones en horarios inapropiados

Escenario: El agente corre a las 2am (hora del servidor) y envia recordatorios.

Estado actual: No hay filtro por horario local del usuario. Las alertas de retraso usan ventanas de tiempo (1h, 4h, 12h despues del retraso) sin considerar la hora del dia.

Problemas detectados:

• Un recordatorio a las 3am es contraproducente.
• Mexico tiene 4 zonas horarias. El servidor podria estar en UTC.
• Los proveedores y usuarios estan en diferentes zonas horarias.

Estrategias propuestas:

• Horario operativo: Solo enviar mensajes proactivos entre 7:00am y 9:00pm hora local del usuario.
• Zona horaria por usuario: Derivar zona horaria del municipio registrado. Default: America/Mexico_City.
• Cola diferida: Si una alerta se genera a las 2am, encolarla para las 7am.

- Excepciones: Alertas criticas de seguridad (fraude, cancelacion de emergencia) se envian siempre.

PROBLEMAS TRANSVERSALES

T.1 Concurrencia y race conditions en la BD

Problema: Multiples background_tasks se ejecutan en paralelo con sesiones de SQLAlchemy potencialmente compartidas.

Impacto: Datos inconsistentes, ordenes duplicadas, presupuestos desbalanceados.

Solucion: Crear nueva sesion de BD por cada background task. Usar SELECT FOR UPDATE en operaciones criticas (presupuestos, aprobaciones, ordenes).

T.2 No hay panel de administracion

Problema: No hay forma de que un humano intervenga en ningun proceso sin acceso directo a la BD.

Impacto: Cuando algo falla (y en produccion SIEMPRE falla), no hay forma rapida de resolverlo.

Solucion: Dashboard web minimo con: ordenes activas, incidencias abiertas, proveedores con problemas, pedidos sin respuesta, aprobaciones pendientes. Con acciones: cancelar orden, cambiar status, contactar usuario, re-cotizar.

T.3 No hay logging centralizado ni alertas

Problema: Los logs van a stdout/stderr. Si el servidor tiene un pico de errores a las 3am, nadie se entera hasta la manana.

Solucion: Integrar con servicio de logging (Datadog, Sentry, CloudWatch). Alertas por Slack/WhatsApp al admin para errores criticos.

T.4 No hay tests automatizados

Problema: Cualquier cambio en el codigo puede romper flujos criticos sin detectarlo.

Solucion: Tests unitarios para cada servicio. Tests de integracion para el flujo completo webhook → cotizacion → orden → entrega. Tests de edge cases documentados en este analisis.

T.5 No hay idempotencia en el webhook

Problema: WhatsApp puede enviar el mismo mensaje multiples veces (retry por timeout). El webhook lo procesaria N veces.

Solucion: Guardar message_id en Redis/BD. Si ya se proceso, ignorar. Ya se llama marcar_como_leido pero eso no previene doble procesamiento.

T.6 Backup y disaster recovery

Problema: Si la BD se corrompe o se pierde, se pierden todos los pedidos, ordenes, cotizaciones, y datos de usuarios.

Solucion: Backups automaticos cada 6 horas. Replicacion de BD. Plan documentado de recuperacion ante desastres.

PRIORIDADES DE IMPLEMENTACION

URGENTE (Bloqueadores de produccion)

1. Notificar al proveedor cuando es elegido (5.1) — Sin esto, el proveedor no sabe que tiene un pedido.
2. Timeout handler de cotizaciones (2.1) — Pedidos pueden quedar en "cotizando" para siempre.
3. Verificar expiradas en el agente proactivo (4.1) — Aprobaciones nunca expiran.
4. Confirmar scheduler del agente proactivo (13.1) — Si no corre, no hay seguimiento.
5. Nueva sesion de BD por background task (T.1) — Race conditions criticas.
6. Implementar cancelacion en cotizando (1.7) — Usuario atrapado si quiere cancelar.

ALTA PRIORIDAD (Primera semana)

7. Debouncing de mensajes (1.6)
8. Confirmacion pre-cotizacion (1.9)
9. Soporte de imagenes para proveedores (2.4)
10. Formato flexible de aprobacion (4.4)
11. Notificacion al proveedor al ser elegido + confirmacion (3.4)
12. Panel admin basico (T.2)

MEDIA PRIORIDAD (Primer mes)

13. Entregas parciales (5.3)
14. Negociacion basica (3.3)
15. Facturacion CFDI (6.6)

16. Horarios de operacion de proveedores (5.11)
17. Deteccion de bloqueo de numero (12.4)
18. Rate limit de mensajes por usuario (13.2)
19. Deteccion de outliers de precios (11.1)
20. Backup y monitoring (T.3, T.6)

BAJA PRIORIDAD (Segundo mes)

21. Soporte de imagenes del usuario (1.3)
22. Credito y pagos parciales (6.4)
23. Negociacion avanzada (3.3)
24. Alertas climaticas (5.7)
25. Fast-track de credit scoring (8.1)
26. Multi-idioma (1.10)

Este documento debe revisarse cada 2 semanas conforme se implementen las estrategias y se descubran nuevos edge cases en produccion.